

Cammini di costo minimo

Dati

- un grafo orientato $G = (N, A)$ con pesi degli archi $c_{i,j}$
($c_{i,j}$: lunghezza dell'arco $(i, j) \in A$ o costo/tempo di attraversamento dell'arco $(i, j) \in A$)
- un nodo origine s e un nodo destinazione t

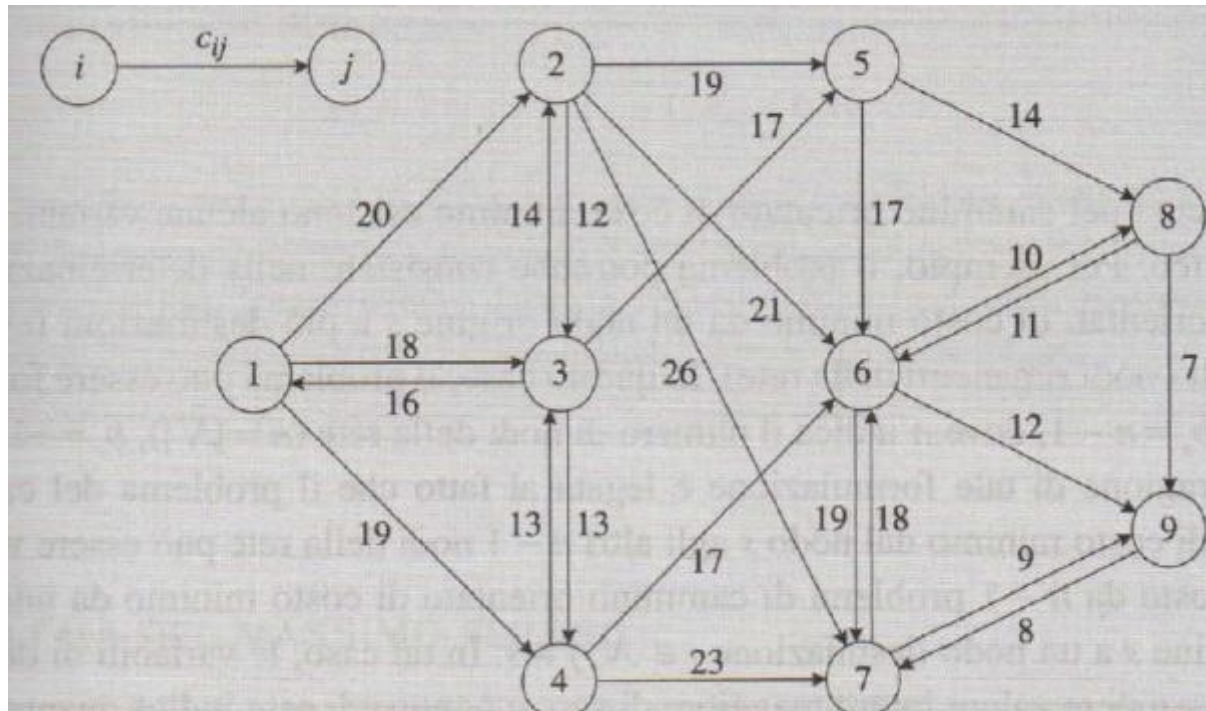
determinare il **cammino di costo minimo** dal nodo origine s al nodo destinazione t , ossia il cammino da s a t per il quale è minima la somma dei pesi degli archi che lo compongono.

Esempio

Rete dei 24 collegamenti orientati (con le rispettive lunghezze in km) che la Endpower può utilizzare per collegare fra loro 9 punti della propria rete di distribuzione del carburante.

Nodo 1: deposito.

Qual è il percorso più breve per inviare un'autocisterna dal deposito all'impianto di distribuzione del carburante situato nel nodo 8.



nodo	stella uscente
1	(1,2), (1,3), (1,4)
2	(2,3), (2,5), (2,6), (2,7)
3	(3,1), (3,2), (3,4), (3,5)
4	(4,3), (4,6), (4,7)
5	(5,6), (5,8)
6	(6,7), (6,8), (6,9)
7	(7,6), (7,9)
8	(8,6), (8,9)
9	(9,7)

$x_{i,j}$: n° unità che fluiscono dal nodo i al nodo j

nel nodo 1 **1** unità entra nella rete dall'esterno

nodo	stella uscente	nodo	flusso entrante	=	flusso uscente
1	(1,2), (1,3), (1,4)	1	$x_{3,1} + \mathbf{1}$	=	$x_{1,2} + x_{1,3} + x_{1,4}$
2	(2,3), (2,5), (2,6), (2,7)	2	$x_{1,2} + x_{3,2}$	=	$x_{2,3} + x_{2,5} + x_{2,6} + x_{2,7}$
3	(3,1), (3,2), (3,4), (3,5)	3	$x_{1,3} + x_{2,3} + x_{4,3}$	=	$x_{3,1} + x_{3,2} + x_{3,4} + x_{3,5}$
4	(4,3), (4,6), (4,7)	4	$x_{1,4} + x_{3,4}$	=	$x_{4,3} + x_{4,6} + x_{4,7}$
5	(5,6), (5,8)	5	$x_{2,5} + x_{3,5}$	=	$x_{5,6} + x_{5,8}$
6	(6,7), (6,8), (6,9)	6	$x_{2,6} + x_{4,6} + x_{5,6} + x_{7,6} + x_{8,6}$	=	$x_{6,7} + x_{6,8} + x_{6,9}$
7	(7,6), (7,9)	7	$x_{2,7} + x_{4,7} + x_{6,7} + x_{9,7}$	=	$x_{7,6} + x_{7,9}$
8	(8,6), (8,9)	8	$x_{5,8} + x_{6,8}$	=	$x_{8,6} + x_{8,9} + \mathbf{1}$
9	(9,7)	9	$x_{6,9} + x_{7,9} + x_{8,9}$	=	$x_{9,7}$

$x_{i,j}$: n° unità che fluiscono dal nodo i al nodo j

dal nodo 8 **1** unità esce dalla rete verso l'esterno

Vincoli di conservazione del flusso ai nodi:

x_{3-1}	$-x_{1-2} - x_{1-3} - x_{1-4}$	$= -1$
$x_{1-2} + x_{3-2}$	$-x_{2-3} - x_{2-5} - x_{2-6} - x_{2-7}$	$= 0$
$x_{1-3} + x_{2-3} + x_{4-3}$	$-x_{3-1} - x_{3-2} - x_{3-4} - x_{3-5}$	$= 0$
$x_{1-4} + x_{3-4}$	$-x_{4-3} - x_{4-6} - x_{4-7}$	$= 0$
$x_{2-5} + x_{3-5}$	$-x_{5-6} - x_{5-8}$	$= 0$
$x_{2-6} + x_{4-6} + x_{5-6} + x_{7-6} + x_{8-6}$	$-x_{6-7} - x_{6-8} - x_{6-9}$	$= 0$
$x_{2-7} + x_{4-7} + x_{6-7} + x_{9-7}$	$-x_{7-6} - x_{7-9}$	$= 0$
$x_{5-8} + x_{6-8}$	$-x_{8-6} - x_{8-9}$	$= 1$
$x_{6-9} + x_{7-9} + x_{8-9}$	$-x_{9-7}$	$= 0$
somma dei flussi sugli archi della stella entrante nel nodo i	somma dei flussi sugli archi della stella uscente nel nodo i	peso del nodo i

Matrice d'incidenza E

	1-2	1-3	1-4	2-3	2-5	2-6	2-7	3-1	3-2	3-4	3-5	4-3	4-6	4-7	5-6	5-8	6-7	6-8	6-9	7-6	7-9	8-6	8-9	9-7
1	-1	-1	-1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	-1	-1	-1	-1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	1	0	1	0	0	0	-1	-1	-1	-1	1	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	1	0	0	0	0	0	0	1	0	-1	-1	-1	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	1	0	0	0	0	0	1	0	0	0	-1	-1	0	0	0	0	0	0	0	0
6	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	0	-1	-1	-1	1	0	1	0	0
7	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	-1	-1	0	0	1
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	-1	-1	0
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	-1

Pesi dei nodi b_i (vettore $\mathbf{b}$)	nodo i	1	2	3	4	5	6	7	8	9
peso b_i		-1	0	0	0	0	0	0	1	0

Vettore dei flussi sugli archi: $\mathbf{x}$

Vincoli di bilancio ai nodi (conservazione del flusso): $\mathbf{E} \mathbf{x} = \mathbf{b}$

Costi degli archi c_{i-j}

1-2	1-3	1-4	2-3	2-5	2-6	2-7	3-1	3-2	3-4	3-5	4-3	4-6	4-7	5-6	5-8	6-7	6-8	6-9	7-6	7-9	8-6	8-9	9-7
20	18	19	12	19	21	26	16	14	13	17	13	17	23	17	14	19	10	12	18	9	11	7	8

Obiettivo: $\min \mathbf{c}^T \mathbf{x}$

Cammini minimi da nodo radice 1 a tutti gli altri nodi:

pesi dei nodi b_i

nodo i	1	2	3	4	5	6	7	8	9
peso b_i	-8	1	1	1	1	1	1	1	1

3 algoritmi

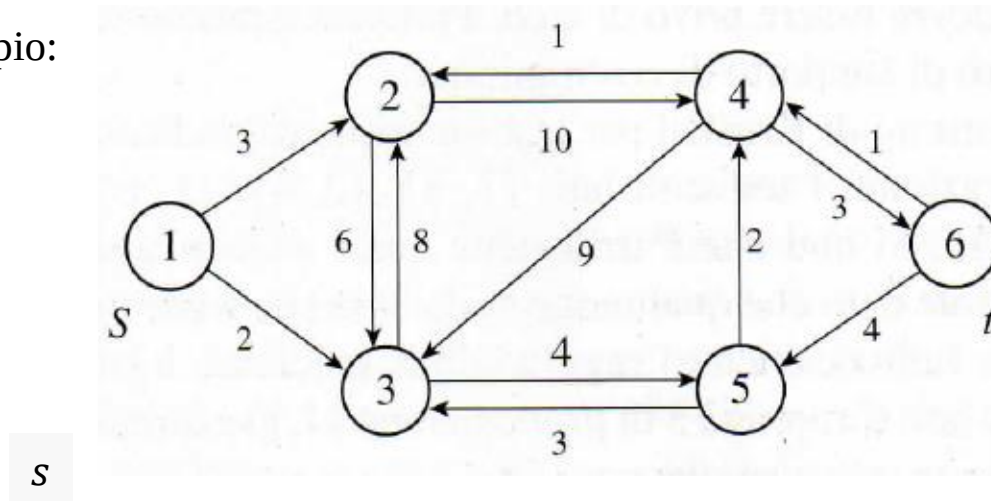
- grafi con presenza di cicli e $c_{i,j} \geq 0, \forall (i,j) \in A$
- grafi aciclici (numerazione topologica dei nodi)
- grafi con presenza di cicli e con $c_{i,j} < 0$ per qualche $(i,j) \in A$

Algoritmo di Dijkstra

Condizione di utilizzo: tutti gli archi hanno **costo nonnegativo**

$$c_{i,j} \geq 0 \quad \forall (i,j) \in A$$

Esempio:



Determinare il cammino minimo

- dal nodo $s = 1$ al nodo $t = 6$
- dal nodo $s = 1$ a tutti gli altri nodi

Insieme S :

- all'inizio contiene il nodo radice s
- ad ogni iterazione si inserisce il nuovo nodo raggiunto mediante un cammino di costo minimo dal nodo radice s

Ogni nodo $i \in S$ per il quale è stato determinato un cammino di costo minimo da s è dotato di un'etichetta con le seguenti due informazioni:

$Pred[i]$ nodo immediatamente precedente il nodo i nel cammino di costo minimo da s

$L[i]$ costo minimo da s ad i

N.B.: in generale possono esistere più cammini distinti di minimo costo

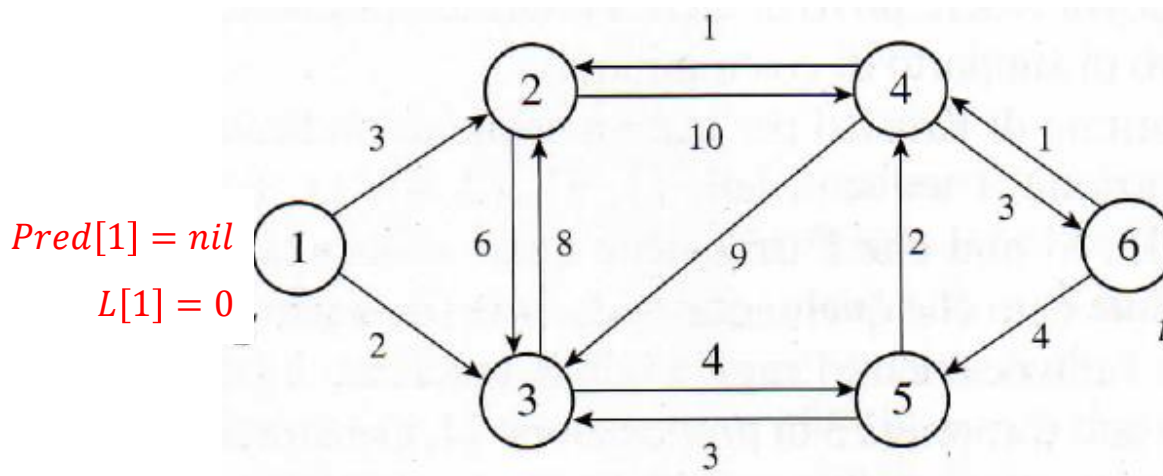
Inizializzazione

➤ l'insieme S contiene solo il nodo radice: $S = \{1\}$

➤ Le etichette del nodo radice sono

$Pred[1] = nil$ (il nodo radice non ha predecessori)

$L[1] = 0$ (il cammino di costo minimo da s a s è nullo)

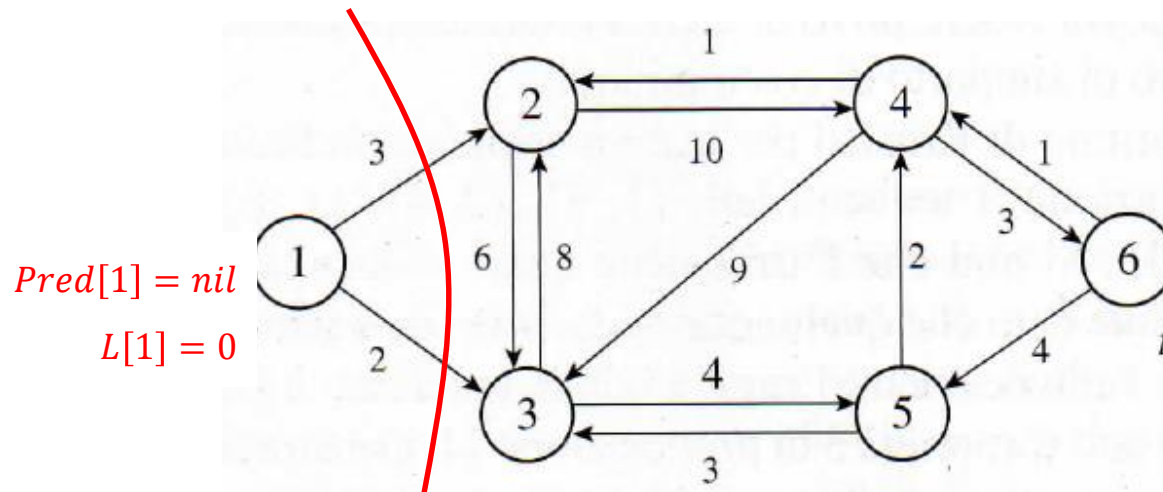


Iterazione 1

Quali nodi sono raggiungibili a partire dai nodi già raggiunti?

Per rispondere a questa domanda si procede come segue:

- Si considera la partizione $(S, N \setminus S)$, detta **taglio**: al Passo 1 il taglio è $S = \{1\}$ e $N \setminus S = \{2, 3, 4, 5, 6\}$

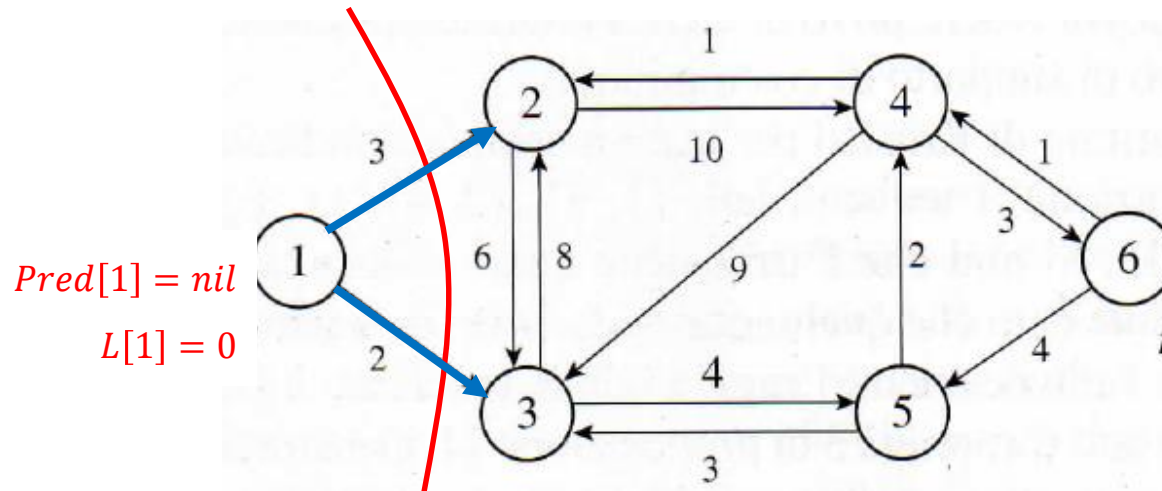


- Si individuano gli archi (i, j) con $i \in S$ e $j \in N \setminus S$, detti **archi diretti del taglio**.

L'insieme degli archi diretti del taglio è indicato con

$$\delta^+(S) = \{(i, j) \in \delta^+(S), i \in S, j \in N \setminus S\}.$$

Al Passo 1 l'**insieme degli archi diretti del taglio** è $\delta^+(S) = \{(1, 2), (1, 3)\}$



L'arco $(1, 2)$ raggiunge il nodo $2 \notin S$ con il cammino $P_2 = \{(1, 2)\}$ di costo $L[1] + c_{1,2} = 3$

L'arco $(1, 3)$ raggiunge il nodo $3 \notin S$ con il cammino $P_3 = \{(1, 3)\}$ di costo $L[1] + c_{1,3} = 2$

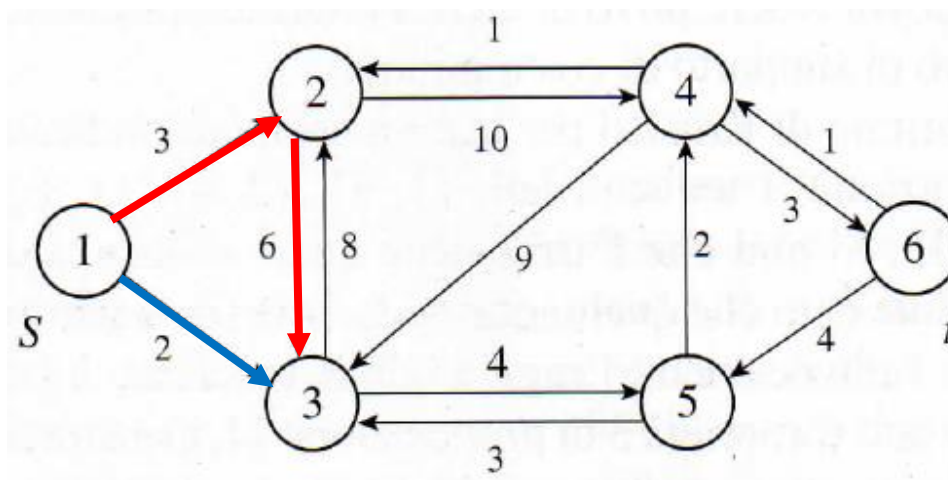
Il cammino di costo minimo tra quelli esaminati in questa iterazione è il cammino dal nodo 1 al nodo 3: **nel grafo non esistono cammini dal nodo 1 al nodo 3 che costano meno di 2**

(come di seguito argomentato)

INFATTI, supponiamo che in una iterazione successiva a quella corrente venga individuato un altro cammino da 1 a 3 e sia k il predecessore del nodo 3 in tale cammino:

il cammino determinato in tale iterazione successiva avrà costo almeno pari a 2: infatti

- il cammino da s a k ha costo ≥ 2 , perché non può essere minore del costo del cammino da s a 3 determinato nell'iterazione corrente (esempio: il **cammino da 1 a $k = 2$** ha costo 3)
- al costo di tale cammino da s a k si deve aggiungere il costo dell'arco $(k, 3)$, che è non negativo per ipotesi (il costo dell'arco $(k, 3)$ è 6)



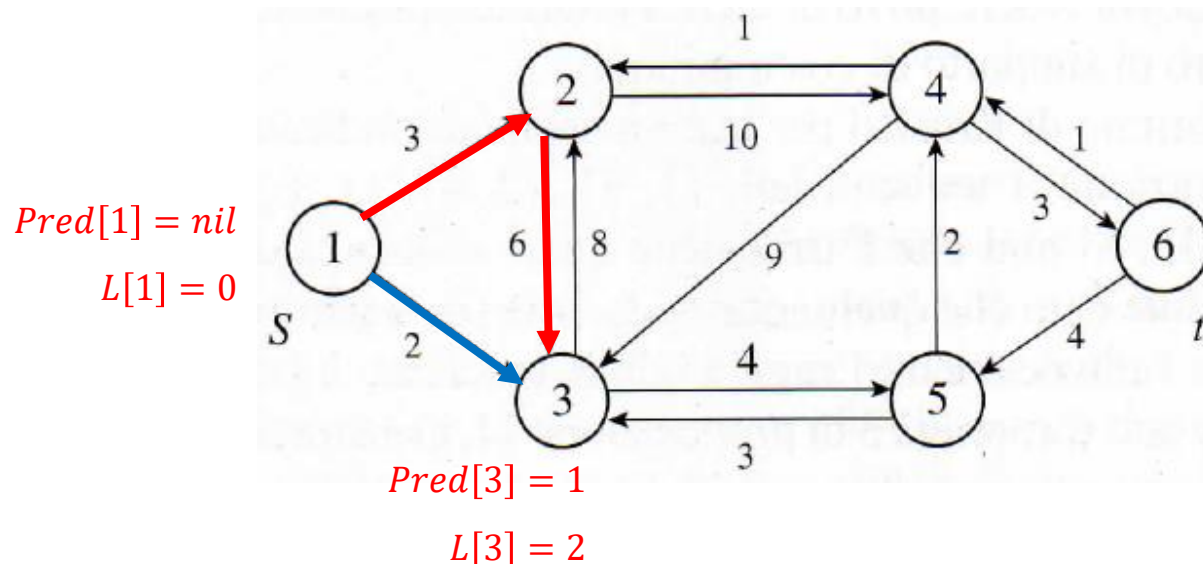
➤ non si può invece escludere che esista un cammino dal nodo 1 al nodo 2 di costo minore di 3

Si aggiornano le informazioni relative al nodo raggiunto dal cammino di costo minimo tra quelli esaminati nell'iterazione corrente.

Nell'iterazione corrente $\min\{L[i] + c_{i,j} : (i,j) \in \delta^+(S)\} = L[1] + c_{1,3}$, quindi

- il nodo **3** viene inserito nell'insieme S : $S = \{1, 3\}$
- $L[3] = L[1] + c_{1,3} = 2$: costo del cammino minimo dall'origine 1 al nodo **3**
- $Pred[3] = 1$: il nodo **1** è il predecessore del nodo **3** in un cammino di costo minimo da s

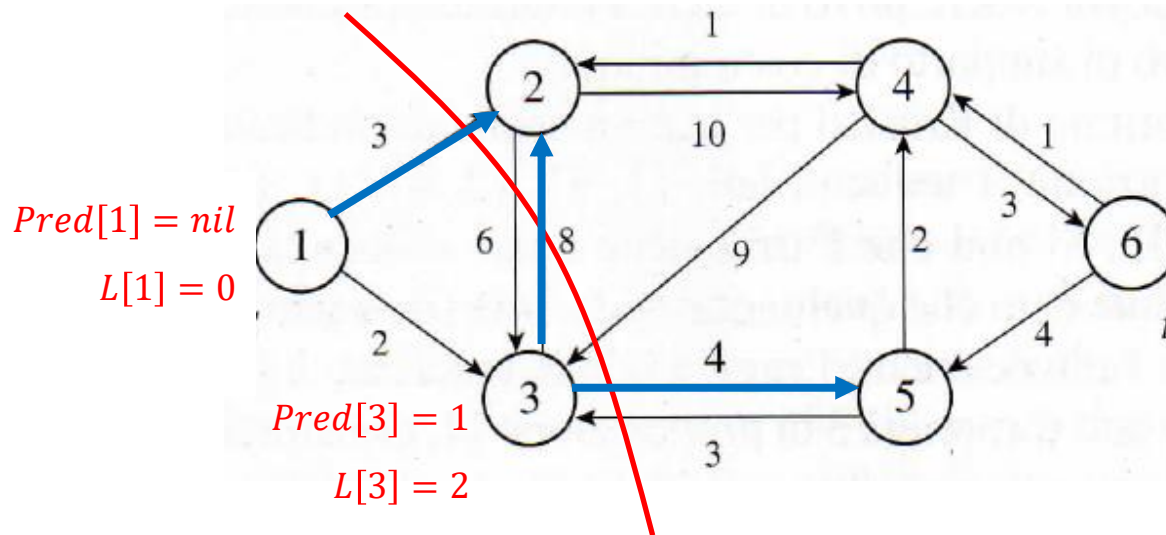
Ciò corrisponde ad aggiungere l'arco **(1, 3)** all'albero dei cammini minimi che collega il nodo s a tutti gli altri nodi



Iterazione 2

All'inizio dell'iterazione 2 il taglio è $S = \{1, 3\}$ e $N \setminus S = \{2, 4, 5, 6\}$.

L'insieme dagli archi diretti del taglio è $\delta^+(S) = \{(1, 2), (3, 2), (3, 5)\}$



Calcoliamo i costi $L[i] + c_{i,j}$ dei cammini da s ai nodi di testa j degli archi $(i, j) \in \delta^+(S)$

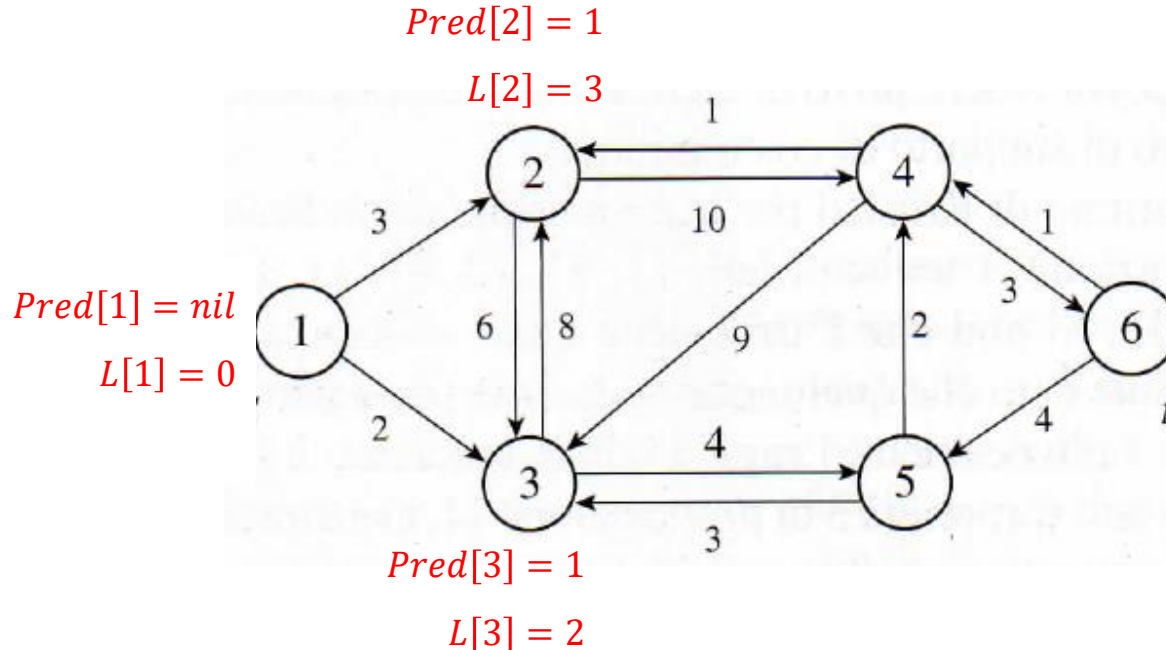
$$(1, 2) \quad L[1] + c_{1,2} = 0 + 3 \quad \leftarrow \text{min}$$

$$(3, 2) \quad L[3] + c_{3,2} = 2 + 8$$

$$(3, 5) \quad L[3] + c_{3,5} = 2 + 4$$

Si sceglie $(v, h) = (1, 2)$ perché $L[1] + c_{1,2} = \min\{L[i] + c_{i,j} : (i, j) \in \delta^+(S)\}$

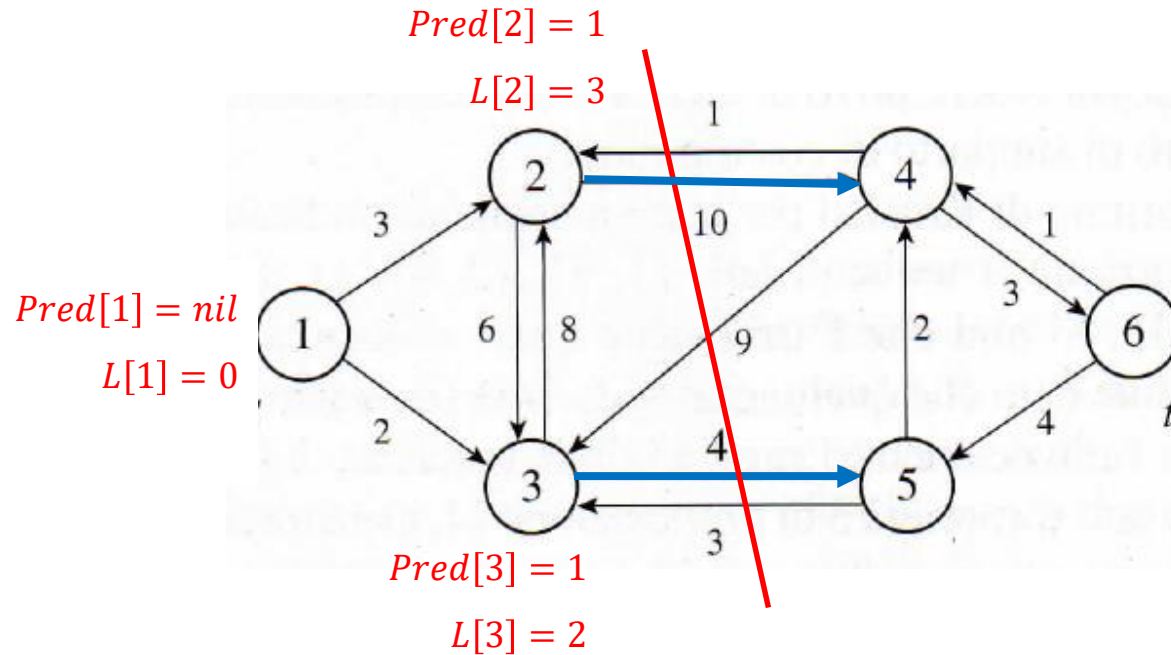
- Al nodo **2**, raggiunto con il cammino di minimo costo tra quelli esaminati nell'iterazione corrente, si assegnano le etichette
- $Pred[h] = 1$: il nodo **1** è il predecessore del nodo **2** nel cammino di costo minimo dal nodo 1
 - $L[h] = L[v] + c_{v,h} = 3$: costo del cammino minimo dal nodo s al nodo **2**
- Il nodo **2** viene inserito nell'insieme S dei nodi etichettati: $S = \{1, 3, 2\}$.



Iterazione 3

All'inizio dell'iterazione 3 il taglio è $S = \{1, 3, 2\}$ e $N \setminus S = \{4, 5, 6\}$

L'insieme dagli archi diretti del taglio è $\delta^+(S) = \{(2, 4), (3, 5)\}$



Calcoliamo i costi $L[i] + c_{i,j}$ dei cammini da s ai nodi di testa j degli archi $(i, j) \in \delta^+(S)$

$$(2, 4) \quad L[2] + c_{2,4} = 3 + 10$$

$$(3, 5) \quad L[3] + c_{3,5} = 2 + 4 \quad \leftarrow \text{min}$$

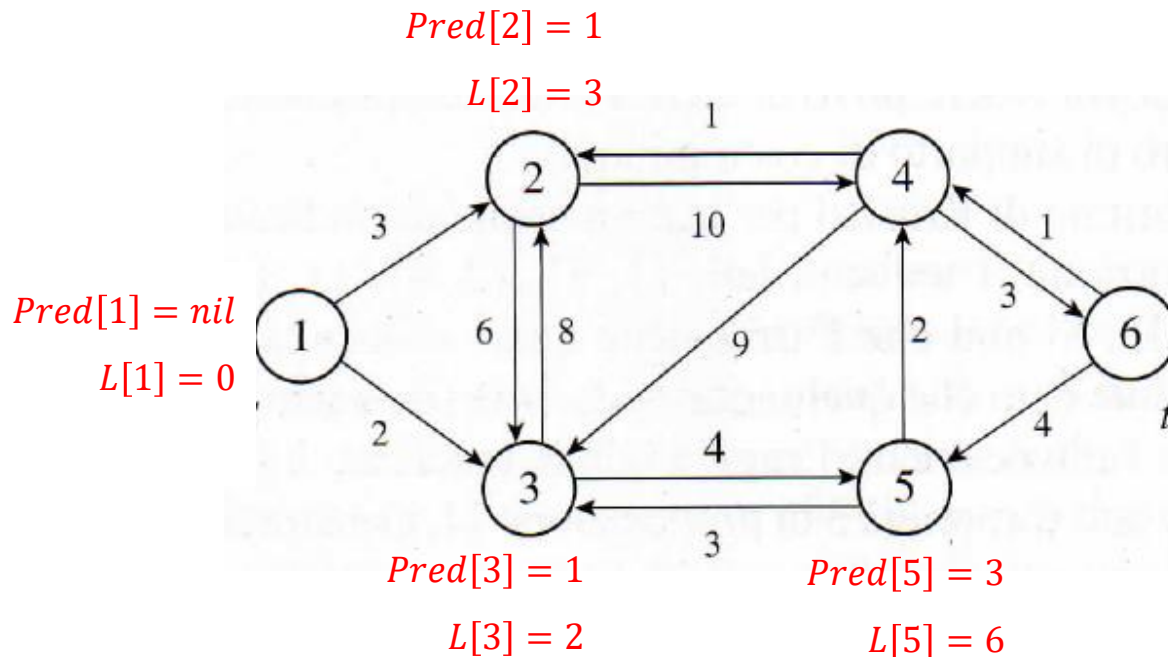
Si sceglie $(v, h) = (3, 5)$ perché $L[3] + c_{3,5} = \min\{L[i] + c_{i,j} : (i, j) \in \delta^+(S)\}$

Al nodo $h = 5$, testa dell'arco scelto $(v, h) = (3, 5)$, si assegnano le etichette

➤ $Pred[h] = v = 1$: il nodo $v = 3$ è il predecessore del nodo $h = 5$ nel cammino di costo minimo dal nodo $s = 1$

➤ $L[h] = L[v] + c_{v,h} = 6$: costo del cammino minimo dal nodo $s = 1$ al nodo $h = 5$

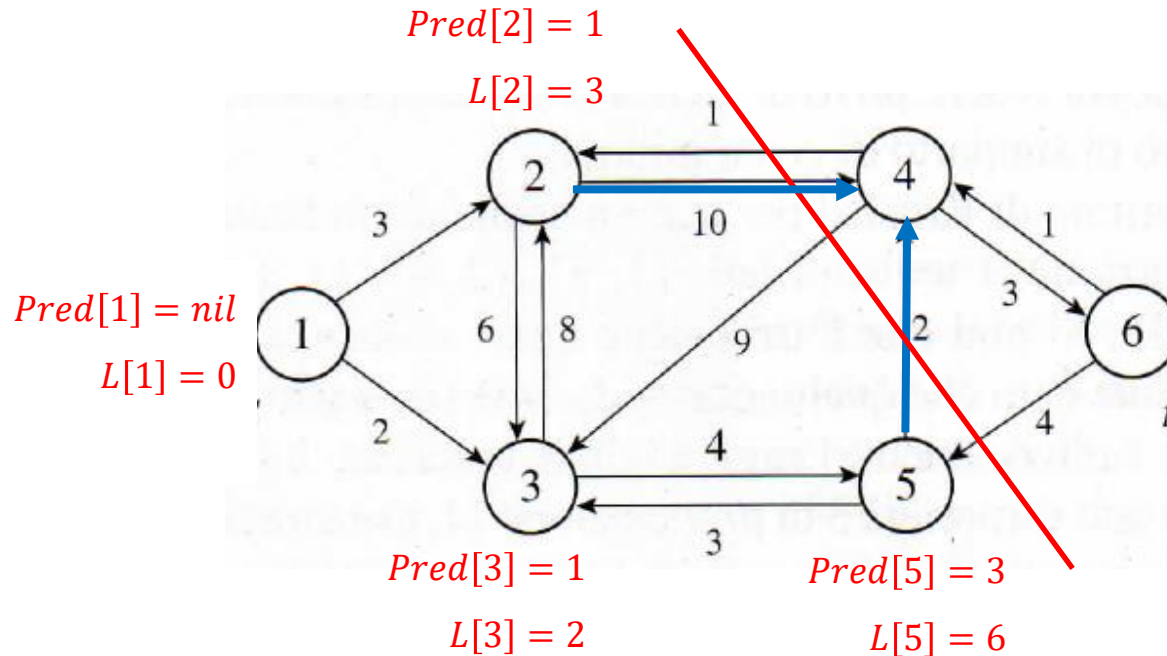
Il nodo $h = 5$ viene inserito nell'insieme S dei nodi etichettati: $S = \{1, 3, 2, 5\}$.



Iterazione 4

All'inizio dell'iterazione 4 il taglio è $S = \{1, 3, 2, 5\}$ e $N \setminus S = \{4, 6\}$

L'insieme dagli archi diretti del taglio è $\delta^+(S) = \{(2, 4), (5, 4)\}$



Calcoliamo i costi $L[i] + c_{i,j}$ dei cammini da s ai nodi di testa j degli archi $(i, j) \in \delta^+(S)$

$$(2, 4) \quad L[2] + c_{2,4} = 3 + 10$$

$$(5, 4) \quad L[5] + c_{5,4} = 6 + 2 \quad \leftarrow \text{min}$$

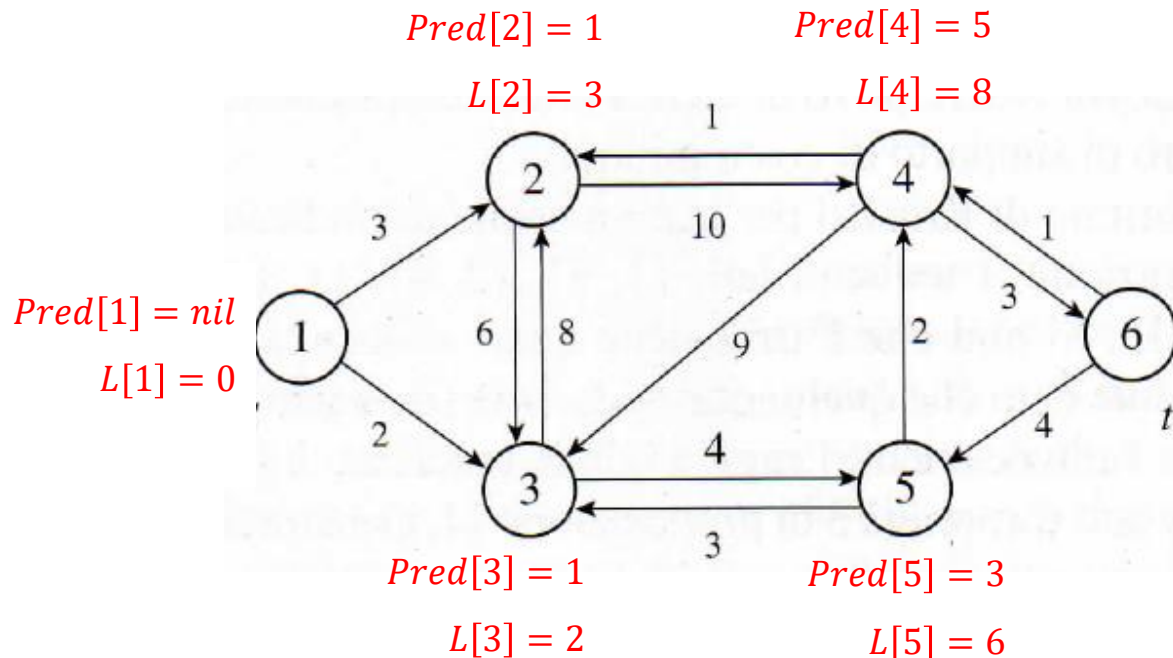
Si sceglie $(v, h) = (5, 4)$ perché $L[5] + c_{5,4} = \min\{L[i] + c_{i,j} : (i, j) \in \delta^+(S)\}$

Al nodo $h = 4$, testa dell'arco scelto $(v, h) = (5, 4)$, si assegnano le etichette

➤ $Pred[h] = v = 5$: il nodo $v = 5$ è il predecessore del nodo $h = 4$ nel cammino di costo minimo dal nodo $s = 1$

➤ $L[h] = L[v] + c_{v,h} = 8$: costo del cammino minimo dal nodo $s = 1$ al nodo $h = 4$

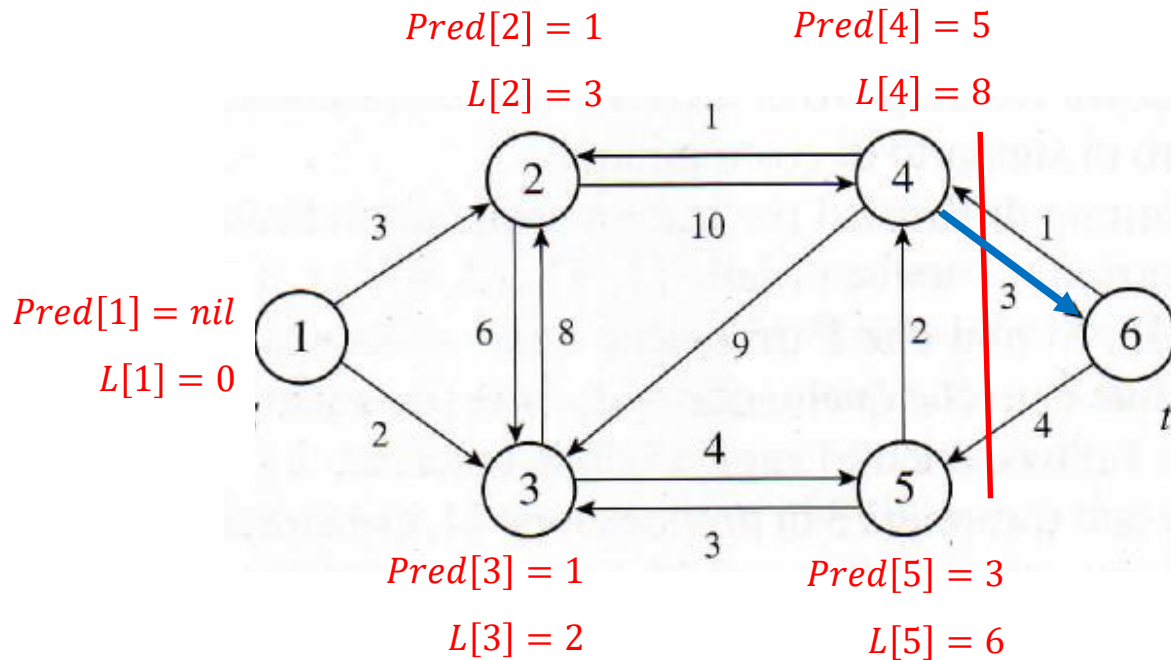
Il nodo $h = 4$ viene inserito nell'insieme S dei nodi etichettati: $S = \{1, 3, 2, 5, 4\}$.



Iterazione 5

All'inizio dell'iterazione 4 il taglio è $S = \{1, 3, 2, 5, 4\}$ e $N \setminus S = \{6\}$

L'insieme dagli archi diretti del taglio è $\delta^+(S) = \{(4, 6)\}$



Calcoliamo i costi $L[i] + c_{i,j}$ dei cammini da s ai nodi di testa j degli archi $(i, j) \in \delta^+(S)$

$$(4, 6) \quad L[4] + c_{4,6} = 8 + 3 \quad \leftarrow \text{min}$$

Al nodo $h = 6$, testa dell'arco scelto $(v, h) = (4, 6)$, si assegnano le etichette

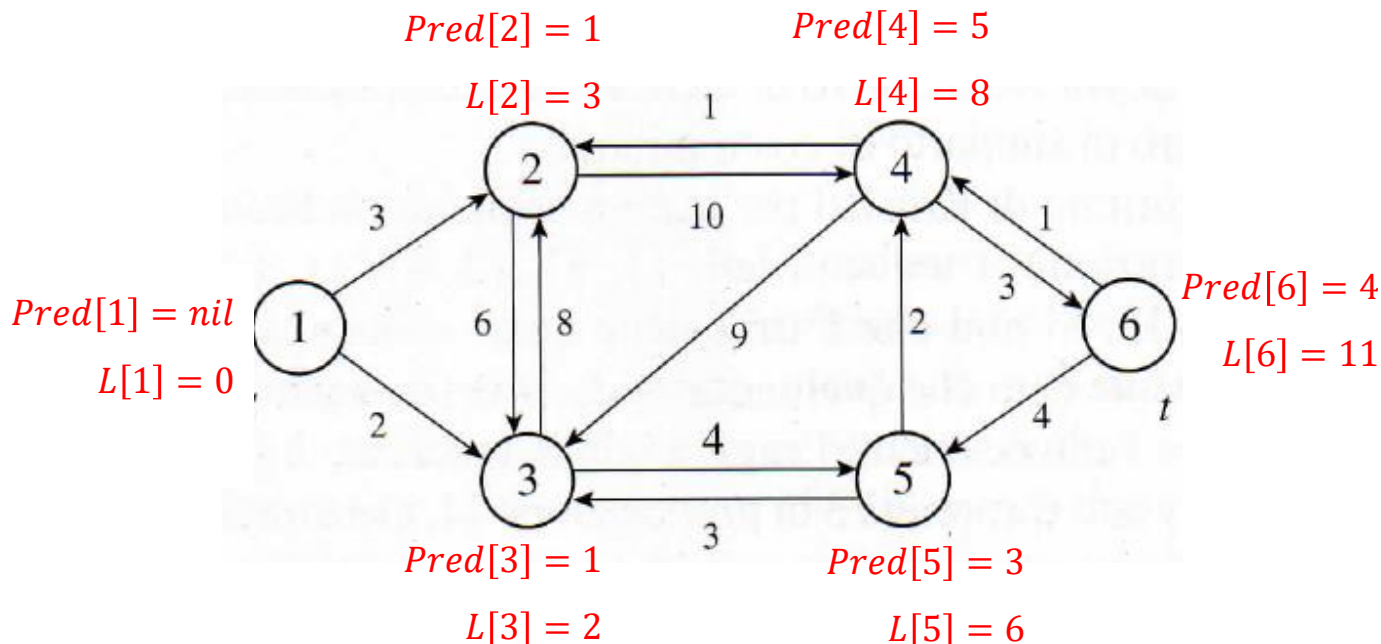
➤ $Pred[h] = v = 4$: il nodo $v = 4$ è il predecessore del nodo $h = 6$ nel cammino di costo minimo dal nodo $s = 1$

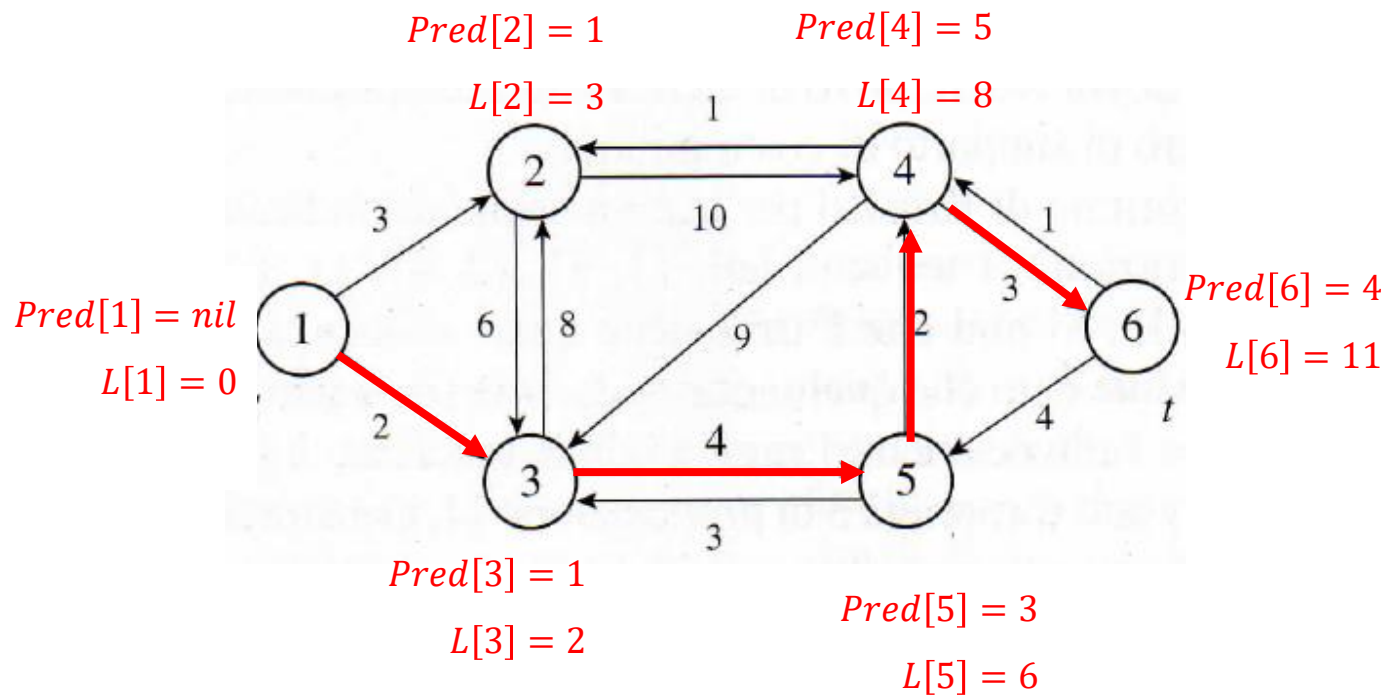
➤ $L[h] = L[v] + c_{v,h} = 11$: costo del cammino minimo dal nodo $s = 1$ al nodo $h = 6$

Il nodo $h = 6$ viene inserito nell'insieme S dei nodi etichettati:

$$S = \{1, 3, 2, 5, 4, 6\} = N.$$

Poiché tutti i nodi sono stati etichettati ($S = N$), l'algoritmo termina.



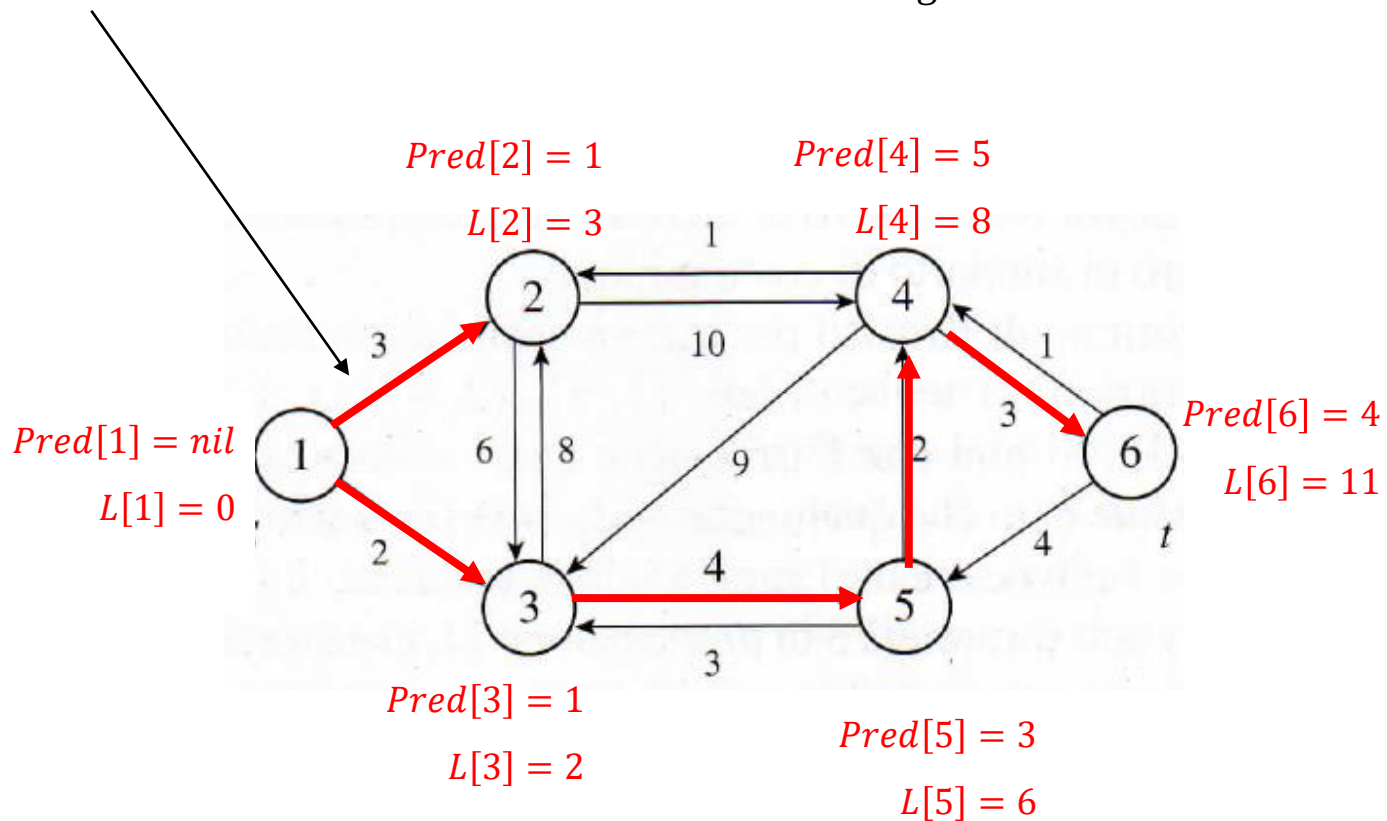


Mediante le etichette $Pred[j]$, partendo dall'ultimo nodo inserito in S , si ricostruisce a ritroso il cammino di costo minimo:

$$Pred[6] = 4 \quad Pred[4] = 5 \quad Pred[5] = 3 \quad Pred[3] = 1$$

Quindi il cammino di costo minimo da $s = 1$ a $t = 6$ è $1 - 3 - 5 - 4 - 6$ di costo 11.

Utilizzando le etichette determinate nel corso dell'algoritmo possiamo determinare l'albero dei cammini di costo minimo da $s = 1$ a tutti gli altri nodi



Pseudocodice dell'algoritmo di Dijkstra

Algoritmo Dijkstra ($N, A, c, s, Pred, L$)

begin

 porre $S = \{s\}, Pred[s] = nil, L[s] = 0$;

while $|S| < n$ **do**

begin

$\forall (i, j) \in \delta^+(S) = \{(i, j) : (i, j) \in A, i \in S, j \notin S\}$ calcolare $L[i] + c_{i,j}$;

 determinare $(v, h) \in \delta^+(S) : L[v] + c_{v,h} = \min\{L[i] + c_{i,j} : (i, j) \in \delta^+(S)\}$;

 porre $Pred[h] = v ; L[h] = L[v] + c_{v,h} ; S = S \cup \{h\}$;

end

end

Algoritmo per grafi orientati aciclici con numerazione topologica dei nodi

Numerazione topologica dei nodi: per ogni $(i, j) \in A$ $i < j$

Il seguente algoritmo determina se il grafo è ciclico o aciclico;

nel caso il grafo sia aciclico, l'algoritmo determina una numerazione topologica dei nodi

Algoritmo Ordinamento Topologico (N, A)

begin

porre $k = 1$;

while $N \neq \emptyset$ **do**

begin

scegliere un nodo $v \in N$ con $BS(v) \neq \emptyset$;

(se un tale nodo non esiste, siamo in presenza di un ciclo $\rightarrow$ STOP)

assegnare il numero k al nodo v ;

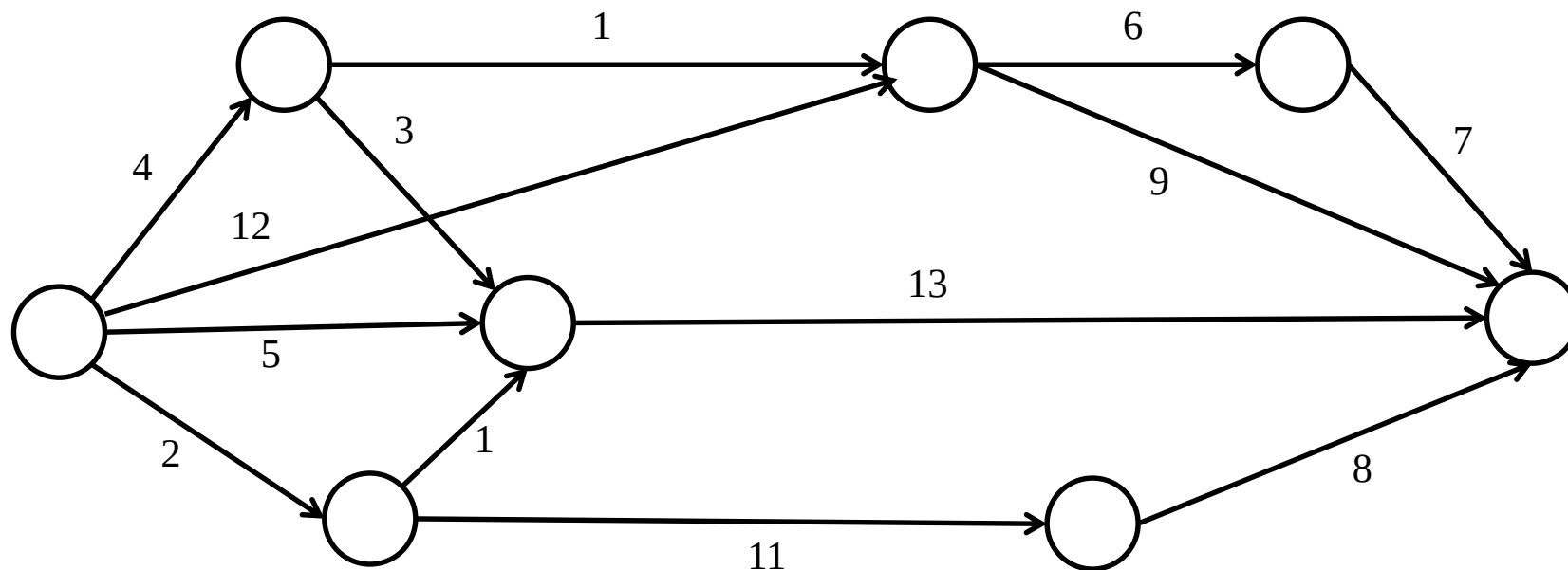
porre $N = N \setminus \{v\}$, $A = A \setminus FS(v)$, $k = k + 1$;

end

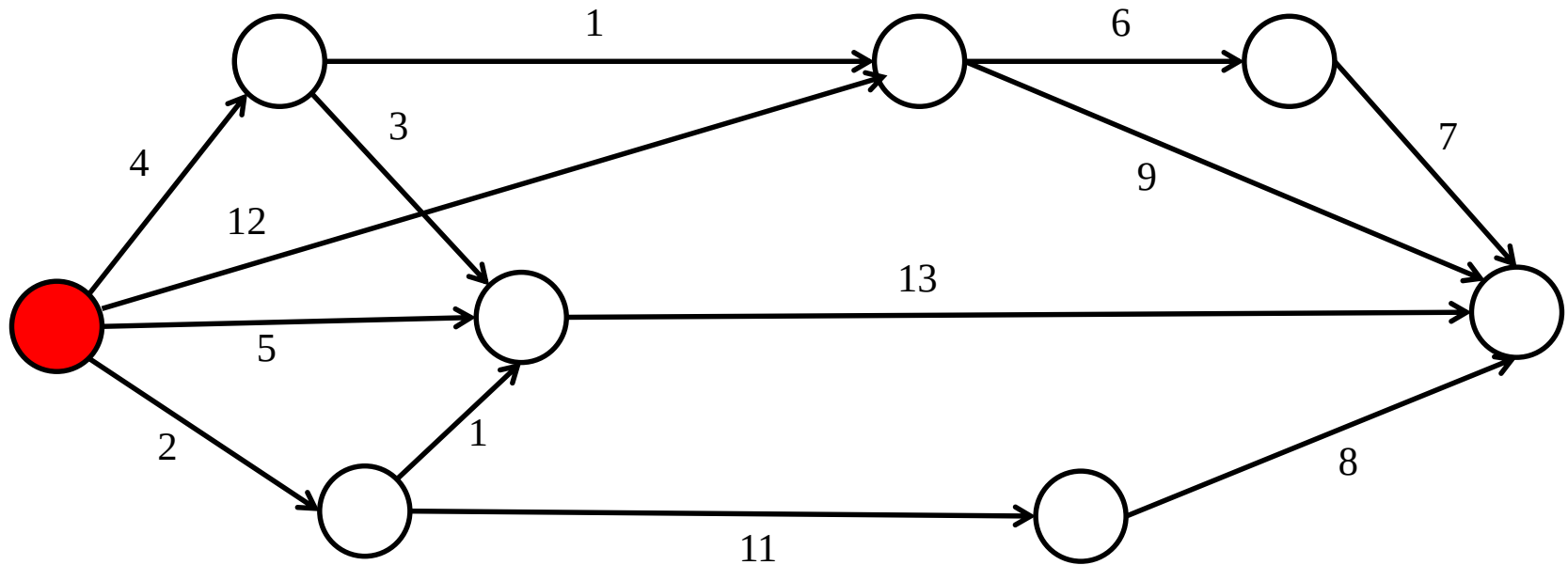
end

Esempio di applicazione dell'Algoritmo Ordinamento Topologico

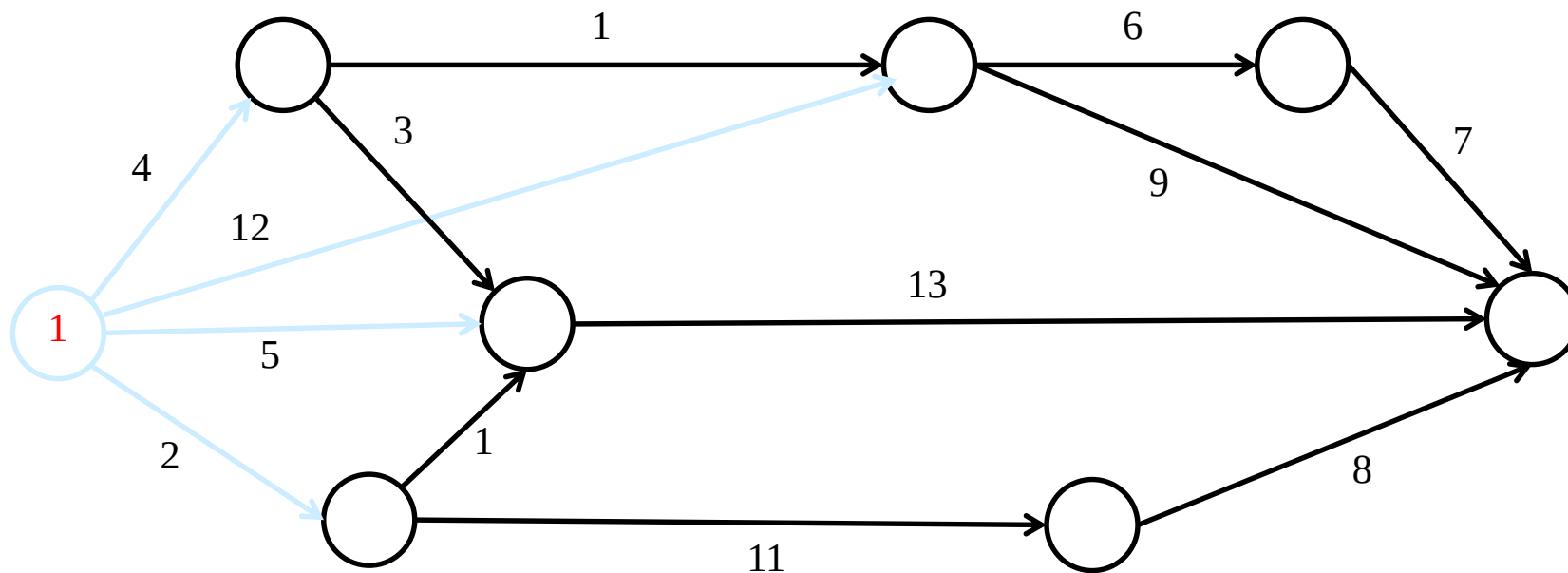
Consideriamo il grafo in figura



Alla prima iterazione viene nominato 1 il nodo evidenziato, essendo l'unico che non ha archi entranti.

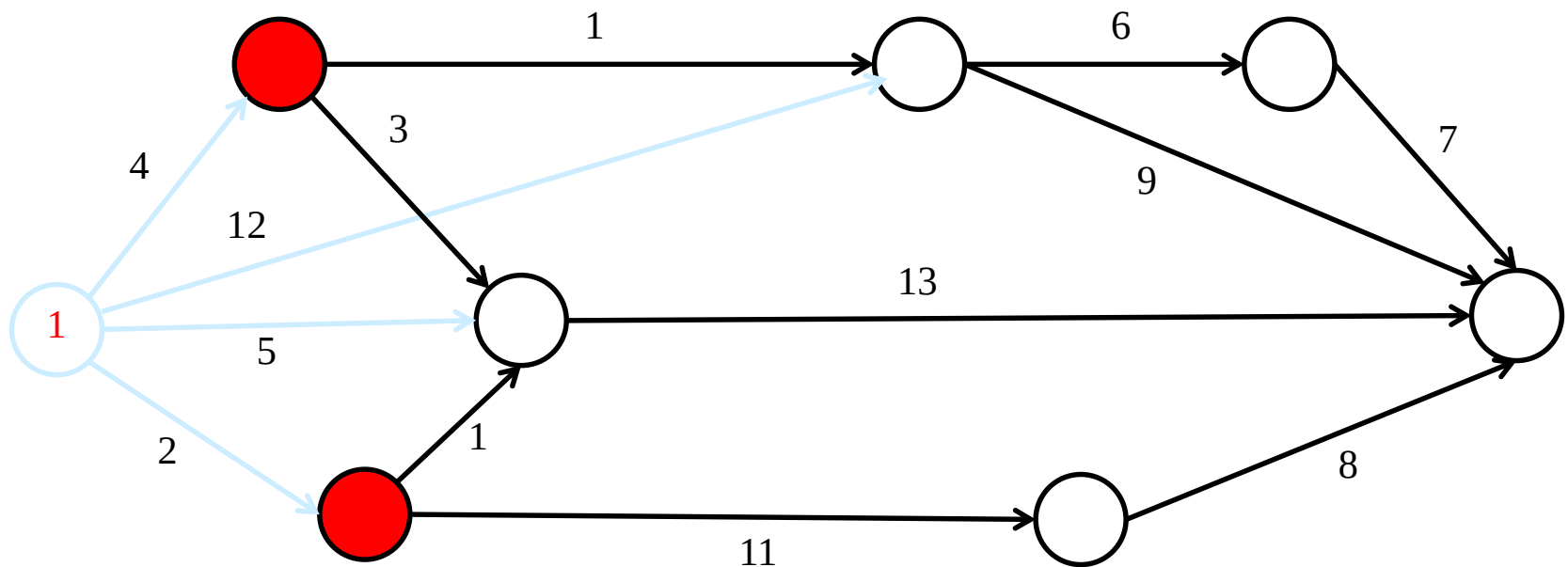


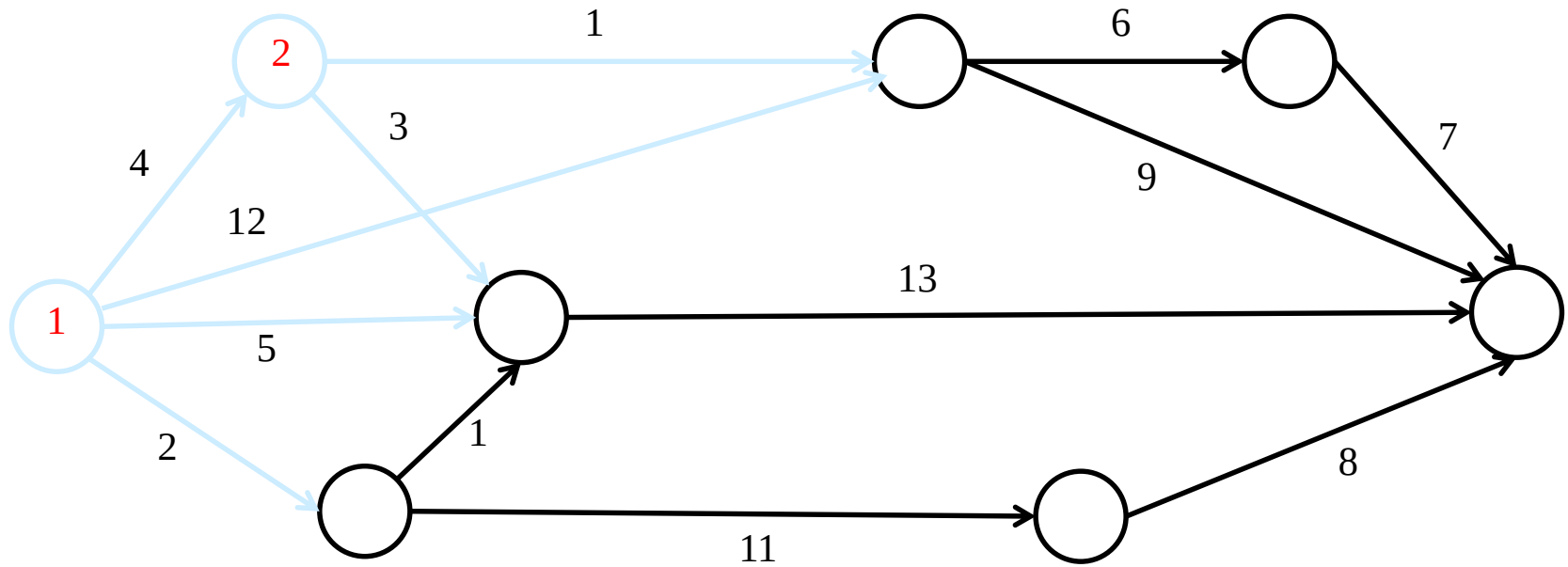
Cancelliamo ora la stella di archi uscenti da tale nodo.

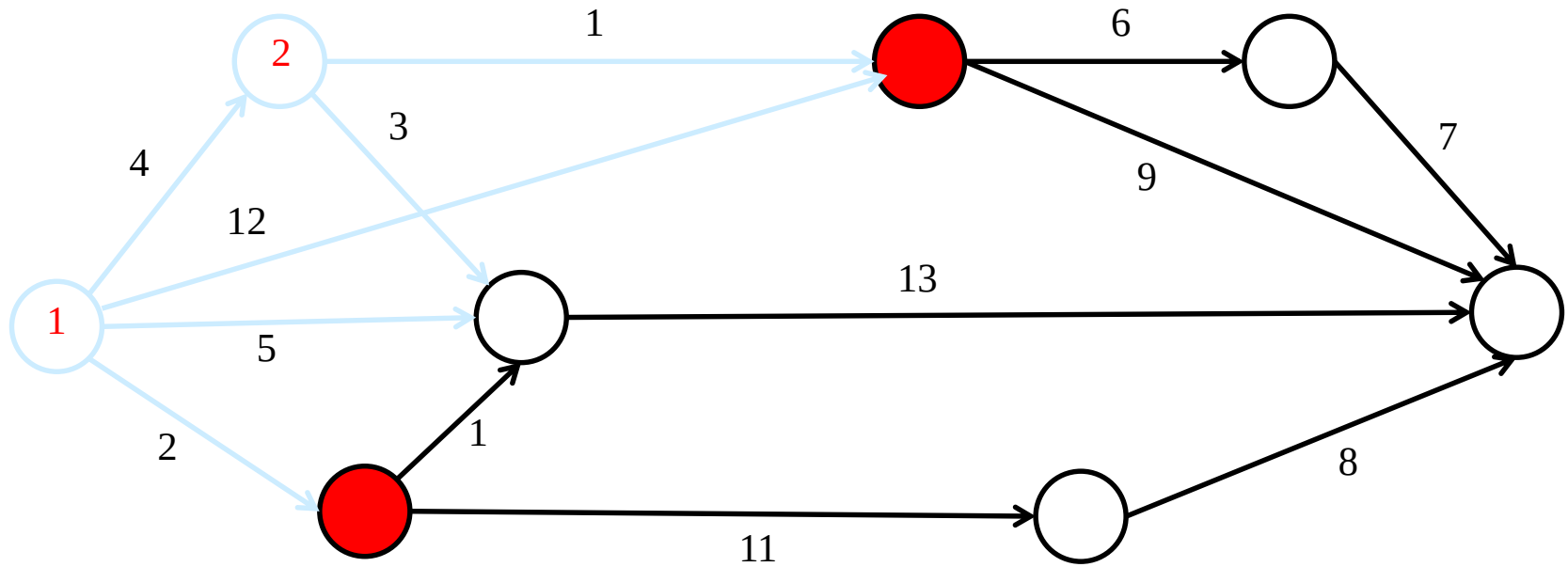


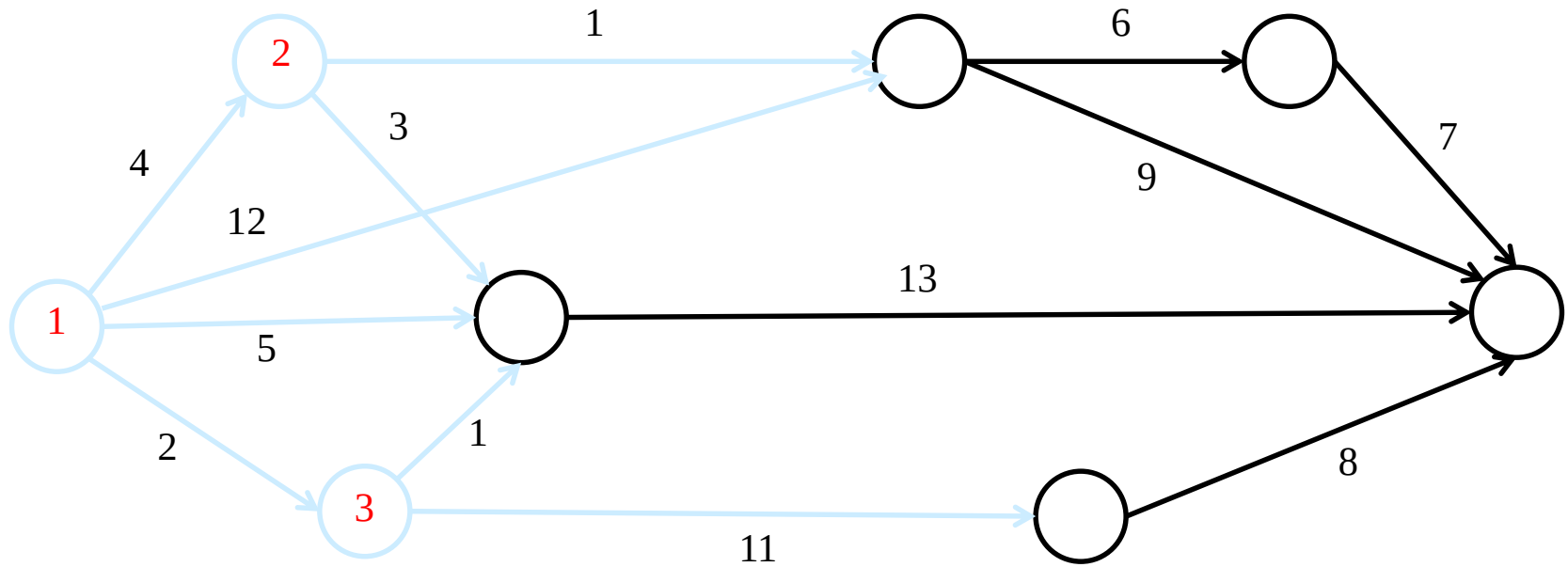
Due nodi non hanno archi entranti:

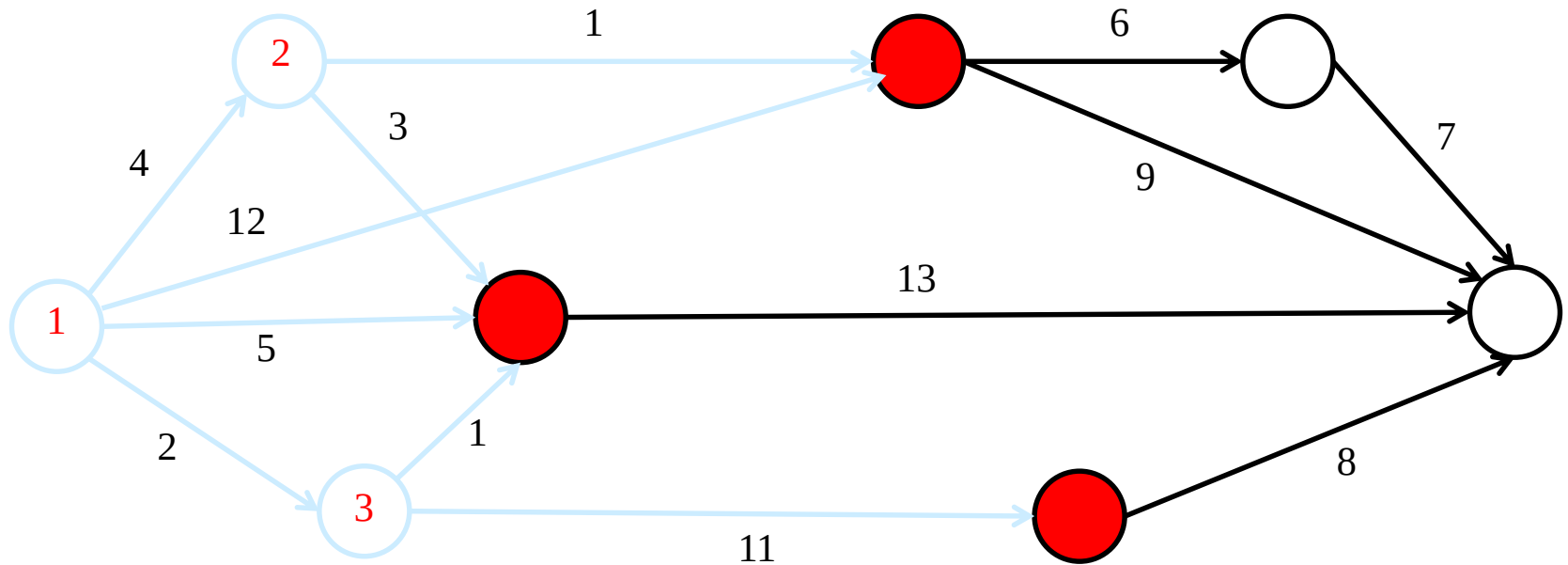
si assegna il numero 2 ad uno dei due nodi scelto arbitrariamente.

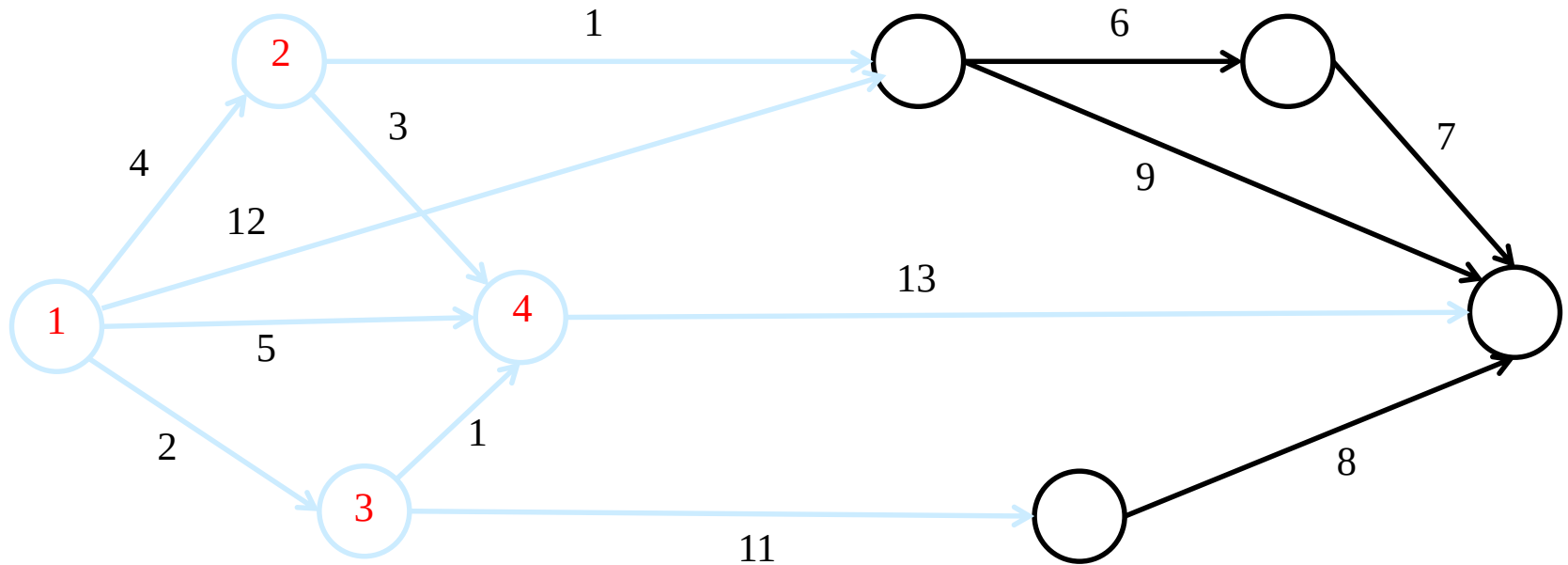


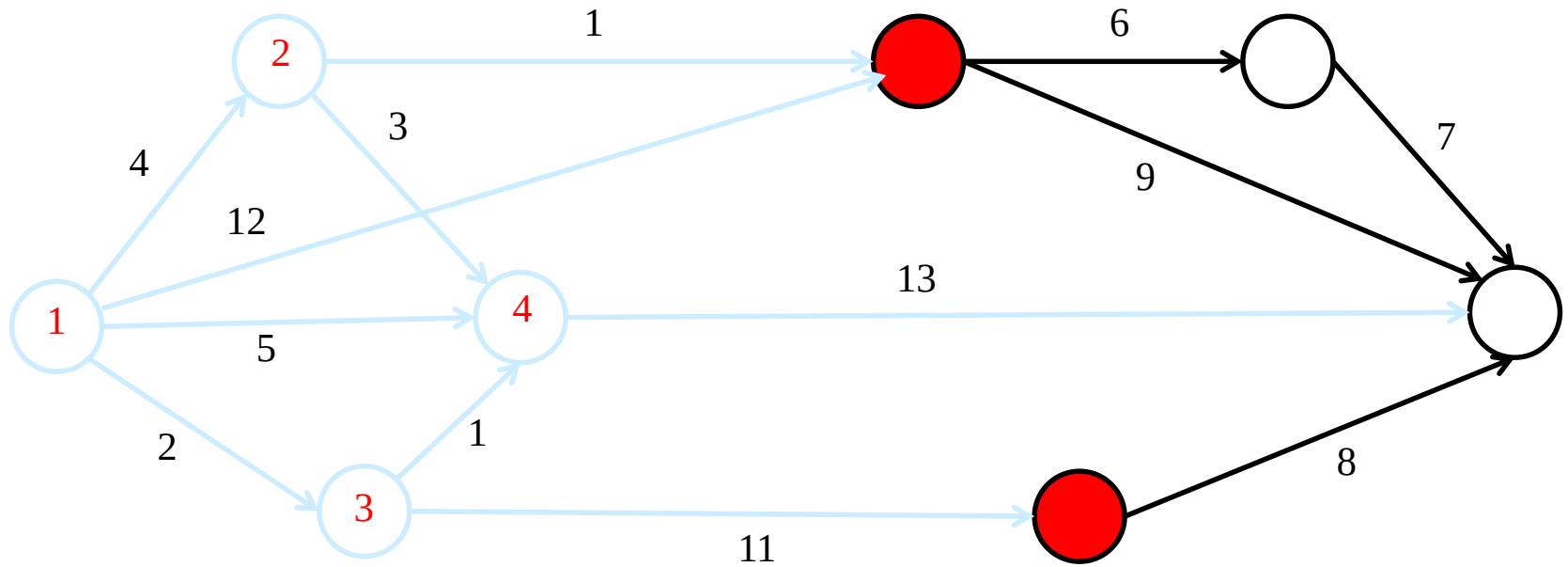


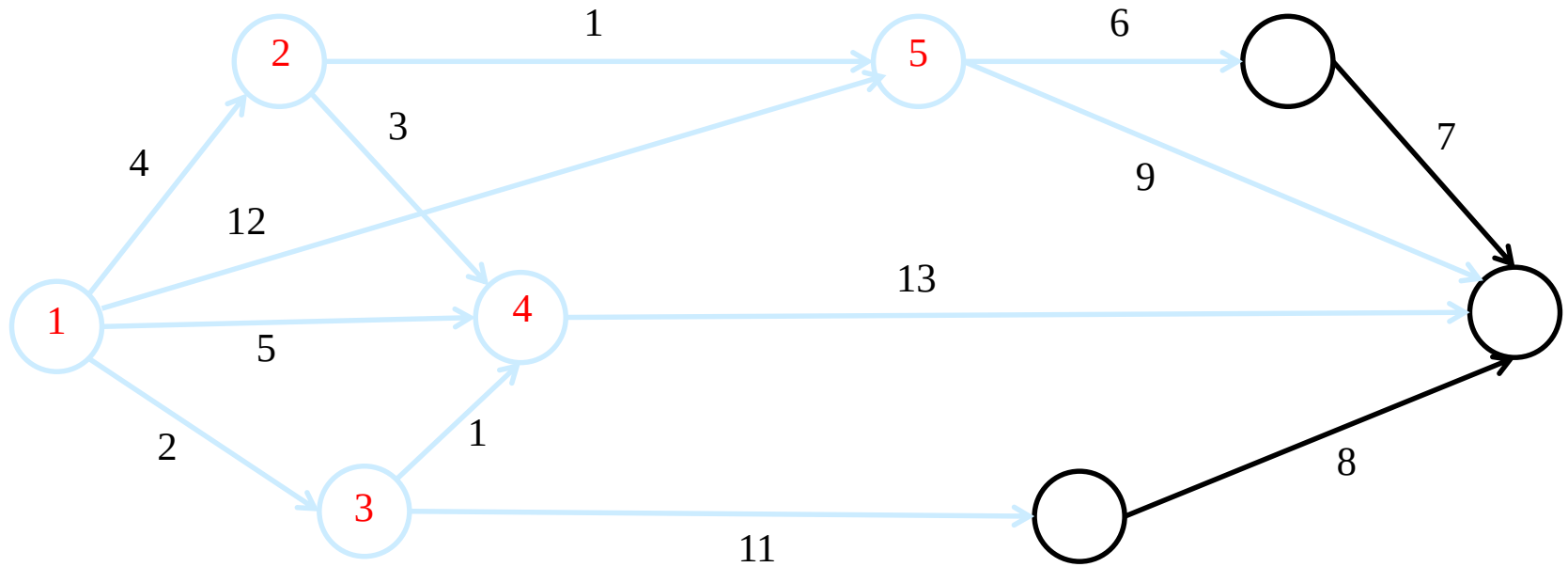


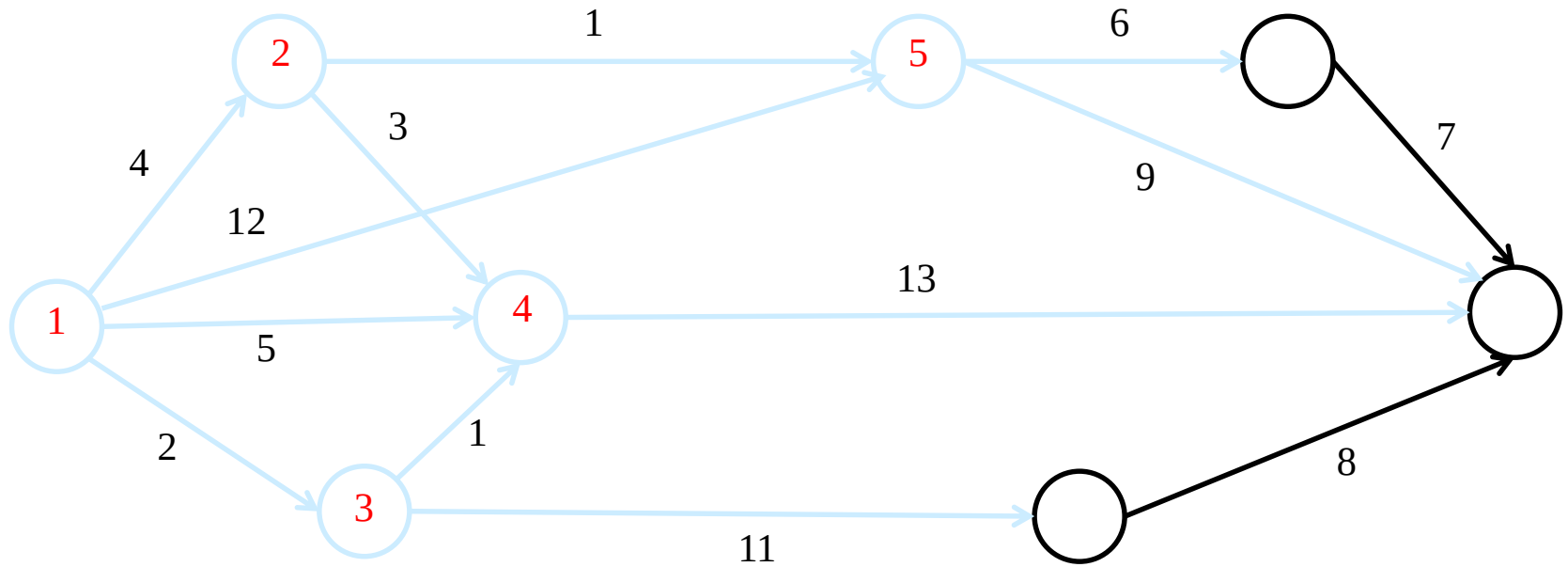


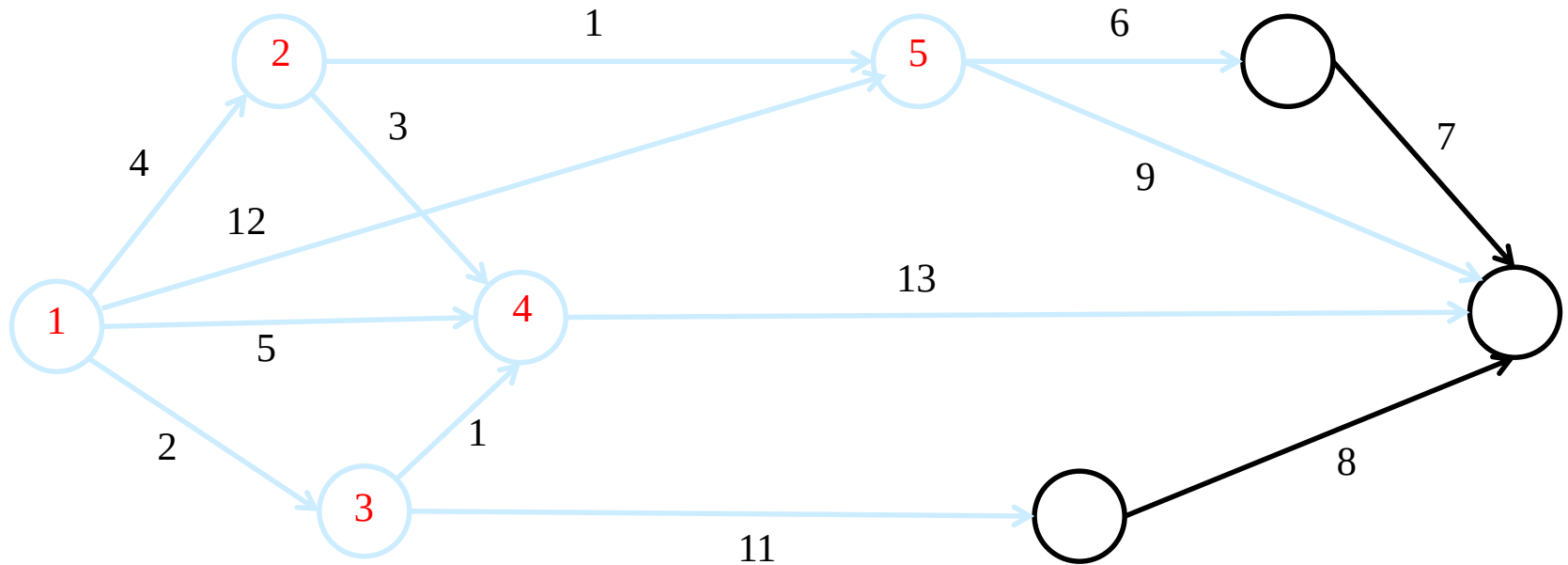


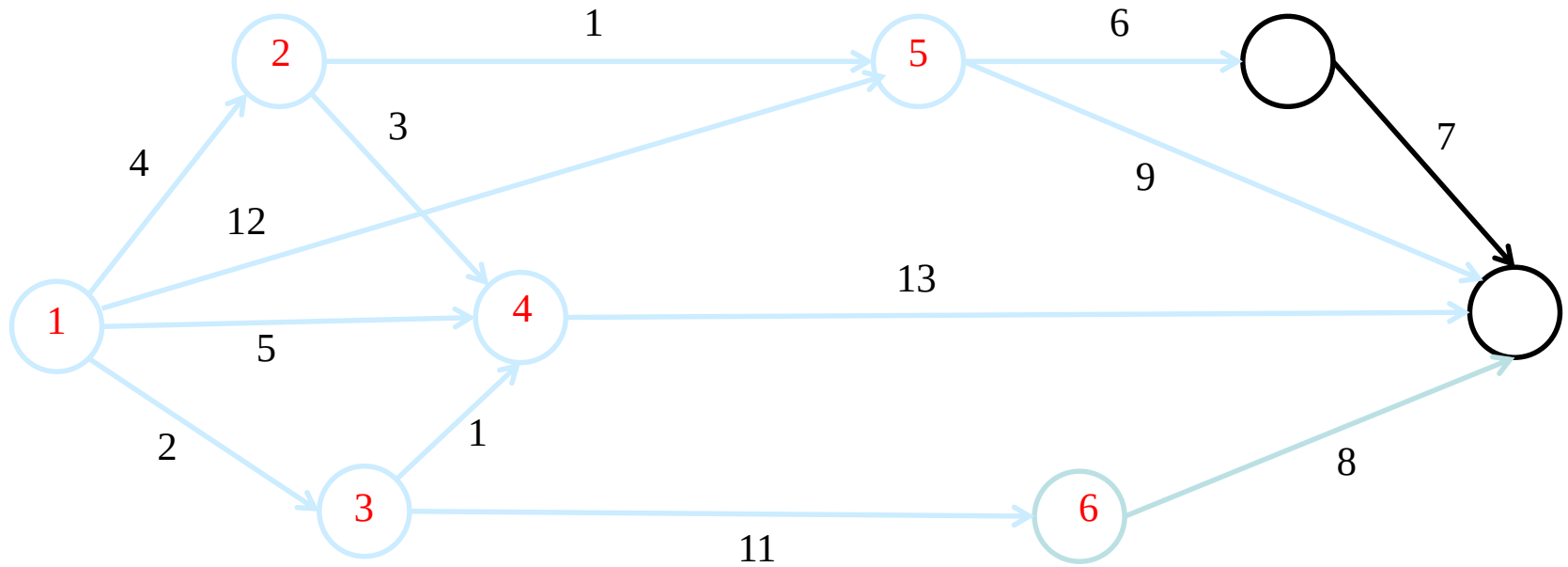


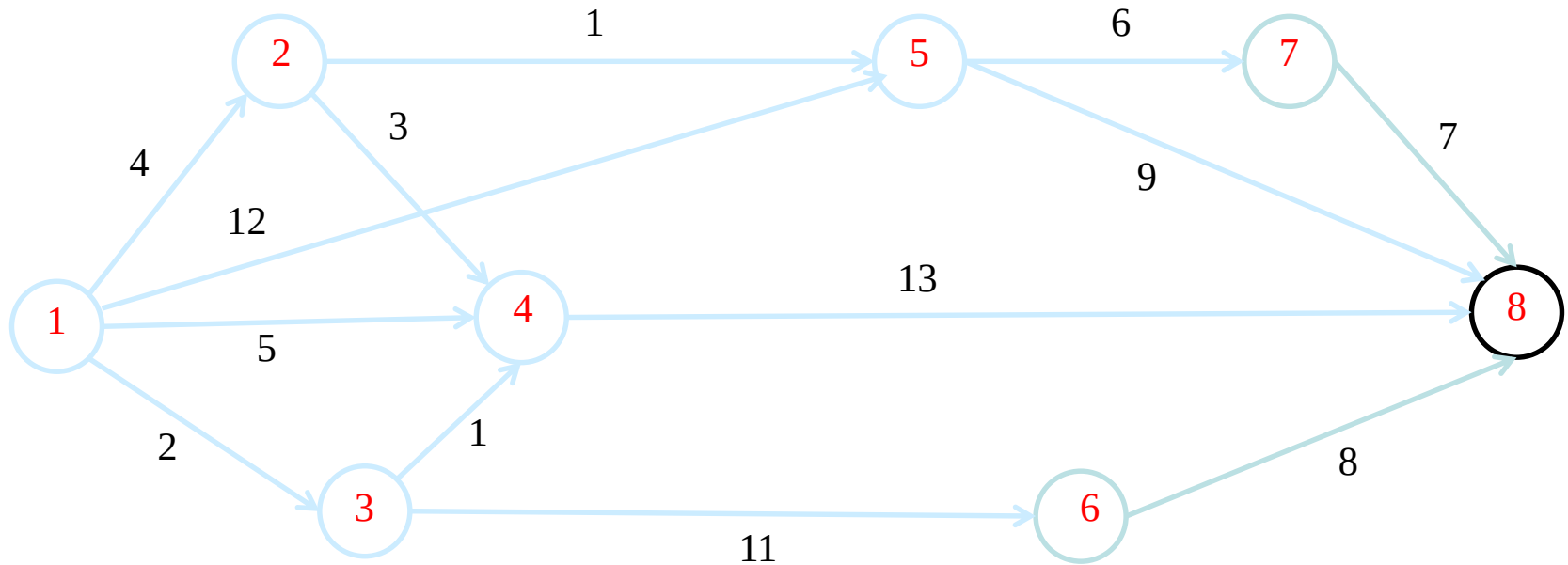












grafo con nodi numerati secondo l'ordinamento topologico:

$$N = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

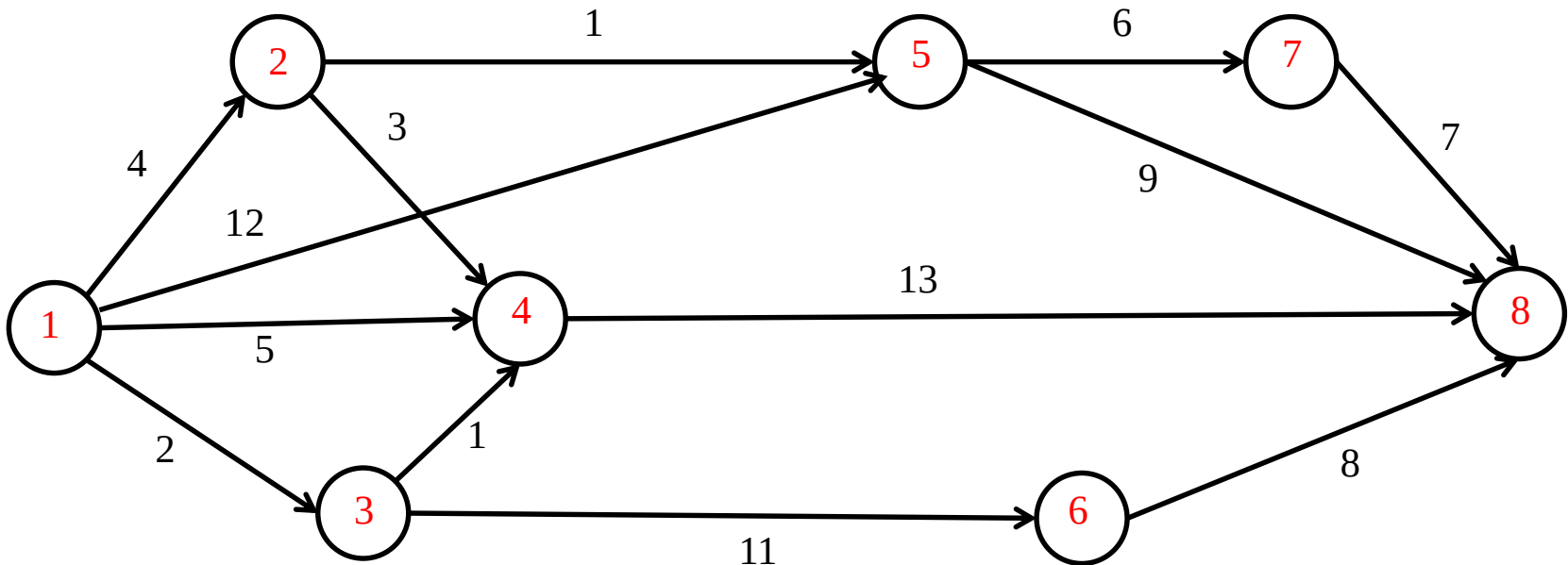
Algoritmo per cammini minimi su grafo aciclico

Determiniamo il cammino di costo minimo dal nodo 1 al nodo 8

nel grafo con i nodi numerati secondo l'ordinamento topologico:

$$N = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$



$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

Inizializzazione

$$Pred[1] = 1$$

$$L[1] = 0$$

Ciclo iterativo:

I nodi vengono esaminati nell'ordine definito dalla numerazione topologica: $i = 2, \dots, n$

Per ogni nodo i si considera la stella entrante $BS(i)$:

- per effetto della numerazione topologica, tutti gli archi entranti (j, i) nel nodo i hanno nodi di coda aventi numeri $j < i$, ossia escono da nodi j per i quali è già stato determinato il costo $L[j]$ del cammino minimo nelle precedenti iterazioni
- si confrontano quindi i costi dei cammini che giungono al nodo corrente i mediante gli archi della stella entrante $BS(i)$ e si sceglie quello di costo minimo

$A = \{(\textcolor{red}{1}, \textcolor{red}{2}), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	$(1, 2)$	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$

$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	$(1, 2)$	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	$(1, 3)$	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	$(1, 2)$	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	$(1, 3)$	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$
$t = 4$	$(1, 4)$	$L[1] + c_{1,4} = 0 + 5 = 5$		
	$(2, 4)$	$L[2] + c_{2,4} = 4 + 3 = 7$		
	$(3, 4)$	$L[3] + c_{3,4} = 2 + 1 = 3$	$Pred[4] = 3$	$L[4] = 3$

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	(1, 2)	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	(1, 3)	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$
$t = 4$	(1, 4)	$L[1] + c_{1,4} = 0 + 5 = 5$	$Pred[4] = 3$	$L[4] = 3$
	(2, 4)	$L[2] + c_{2,4} = 4 + 3 = 7$		
	(3, 4)	$L[3] + c_{3,4} = 2 + 1 = 3$		
$t = 5$	(1, 5)	$L[1] + c_{1,5} = 0 + 12 = 12$	$Pred[5] = 2$	$L[5] = 5$
	(2, 5)	$L[2] + c_{2,5} = 4 + 1 = 5$		

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	(1, 2)	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	(1, 3)	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$
$t = 4$	(1, 4)	$L[1] + c_{1,4} = 0 + 5 = 5$	$Pred[4] = 3$	$L[4] = 3$
	(2, 4)	$L[2] + c_{2,4} = 4 + 3 = 7$		
	(3, 4)	$L[3] + c_{3,4} = 2 + 1 = 3$		
$t = 5$	(1, 5)	$L[1] + c_{1,5} = 0 + 12 = 12$	$Pred[5] = 2$	$L[5] = 5$
	(2, 5)	$L[2] + c_{2,5} = 4 + 1 = 5$		
$t = 6$	(3, 6)	$L[3] + c_{3,6} = 2 + 11 = 13$	$Pred[6] = 3$	$L[6] = 13$

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	(1, 2)	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	(1, 3)	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$
$t = 4$	(1, 4)	$L[1] + c_{1,4} = 0 + 5 = 5$	$Pred[4] = 3$	$L[4] = 3$
	(2, 4)	$L[2] + c_{2,4} = 4 + 3 = 7$		
	(3, 4)	$L[3] + c_{3,4} = 2 + 1 = 3$		
$t = 5$	(1, 5)	$L[1] + c_{1,5} = 0 + 12 = 12$	$Pred[5] = 2$	$L[5] = 5$
	(2, 5)	$L[2] + c_{2,5} = 4 + 1 = 5$		
$t = 6$	(3, 6)	$L[3] + c_{3,6} = 2 + 11 = 13$	$Pred[6] = 3$	$L[6] = 13$
$t = 7$	(5, 7)	$L[5] + c_{5,7} = 5 + 6 = 11$	$Pred[7] = 5$	$L[7] = 11$

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

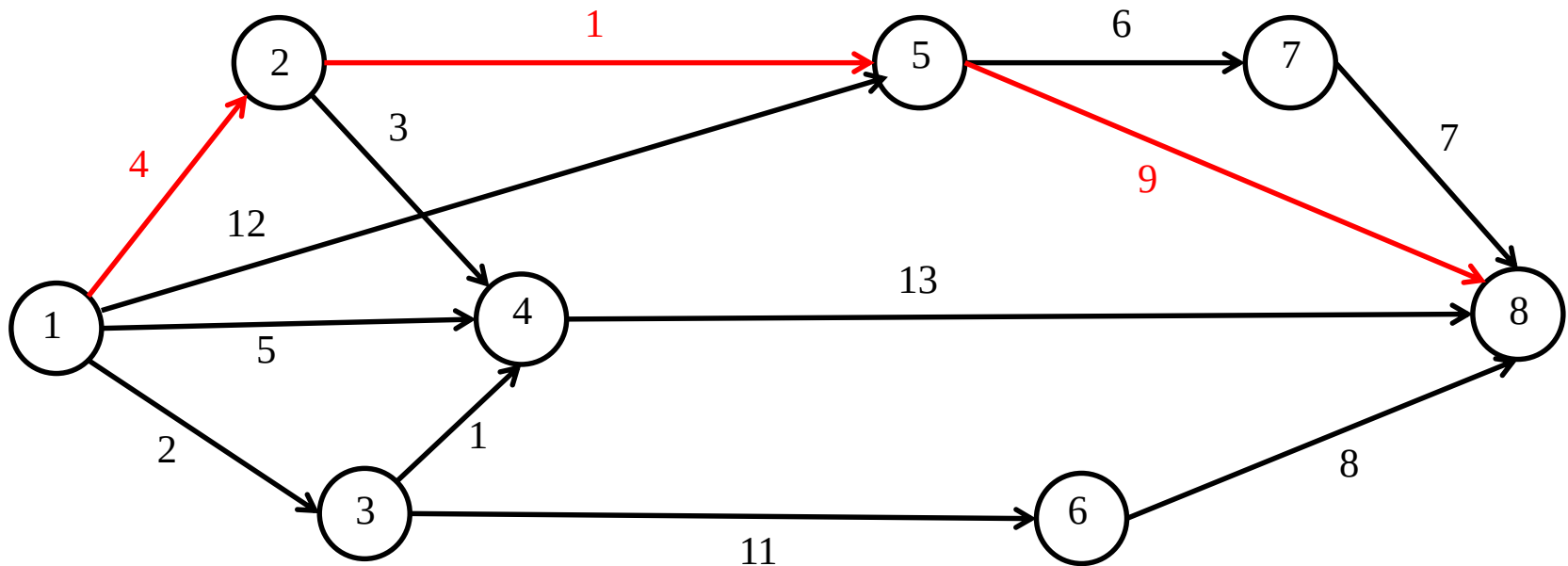
Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	(1, 2)	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	(1, 3)	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$
$t = 4$	(1, 4)	$L[1] + c_{1,4} = 0 + 5 = 5$	$Pred[4] = 3$	$L[4] = 3$
	(2, 4)	$L[2] + c_{2,4} = 4 + 3 = 7$		
	(3, 4)	$L[3] + c_{3,4} = 2 + 1 = 3$		
$t = 5$	(1, 5)	$L[1] + c_{1,5} = 0 + 12 = 12$	$Pred[5] = 2$	$L[5] = 5$
	(2, 5)	$L[2] + c_{2,5} = 4 + 1 = 5$		
$t = 6$	(3, 6)	$L[3] + c_{3,6} = 2 + 11 = 13$	$Pred[6] = 3$	$L[6] = 13$
$t = 7$	(5, 7)	$L[5] + c_{5,7} = 5 + 6 = 11$	$Pred[7] = 5$	$L[7] = 11$
$t = 8$	(4, 8)	$L[4] + c_{4,8} = 3 + 13 = 16$	$Pred[8] = 5$	$L[8] = 14$
	(5, 8)	$L[5] + c_{5,8} = 5 + 9 = 14$		
	(6, 8)	$L[6] + c_{6,8} = 13 + 8 = 21$		
	(7, 8)	$L[7] + c_{7,8} = 11 + 7 = 18$		

$$A = \{(1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 6), (5, 7), (4, 8), (5, 8), (6, 8), (7, 8)\}$$

Inizializzazione			$Pred[1] = 1$	$L[1] = 0$
$t = 2$	(1, 2)	$L[1] + c_{1,2} = 0 + 4 = 4$	$Pred[2] = 1$	$L[2] = 4$
$t = 3$	(1, 3)	$L[1] + c_{1,3} = 0 + 2 = 2$	$Pred[3] = 1$	$L[3] = 2$
$t = 4$	(1, 4)	$L[1] + c_{1,4} = 0 + 5 = 5$	$Pred[4] = 3$	$L[4] = 3$
	(2, 4)	$L[2] + c_{2,4} = 4 + 3 = 7$		
	(3, 4)	$L[3] + c_{3,4} = 2 + 1 = 3$		
$t = 5$	(1, 5)	$L[1] + c_{1,5} = 0 + 12 = 12$	$Pred[5] = 2$	$L[5] = 5$
	(2, 5)	$L[2] + c_{2,5} = 4 + 1 = 5$		
$t = 6$	(3, 6)	$L[3] + c_{3,6} = 2 + 11 = 13$	$Pred[6] = 3$	$L[6] = 13$
$t = 7$	(5, 7)	$L[5] + c_{5,7} = 5 + 6 = 11$	$Pred[7] = 5$	$L[7] = 11$
$t = 8$	(4, 8)	$L[4] + c_{4,8} = 3 + 13 = 16$	$Pred[8] = 5$	$L[8] = 14$
	(5, 8)	$L[5] + c_{5,8} = 5 + 9 = 14$		
	(6, 8)	$L[6] + c_{6,8} = 13 + 8 = 21$		
	(7, 8)	$L[7] + c_{7,8} = 11 + 7 = 18$		

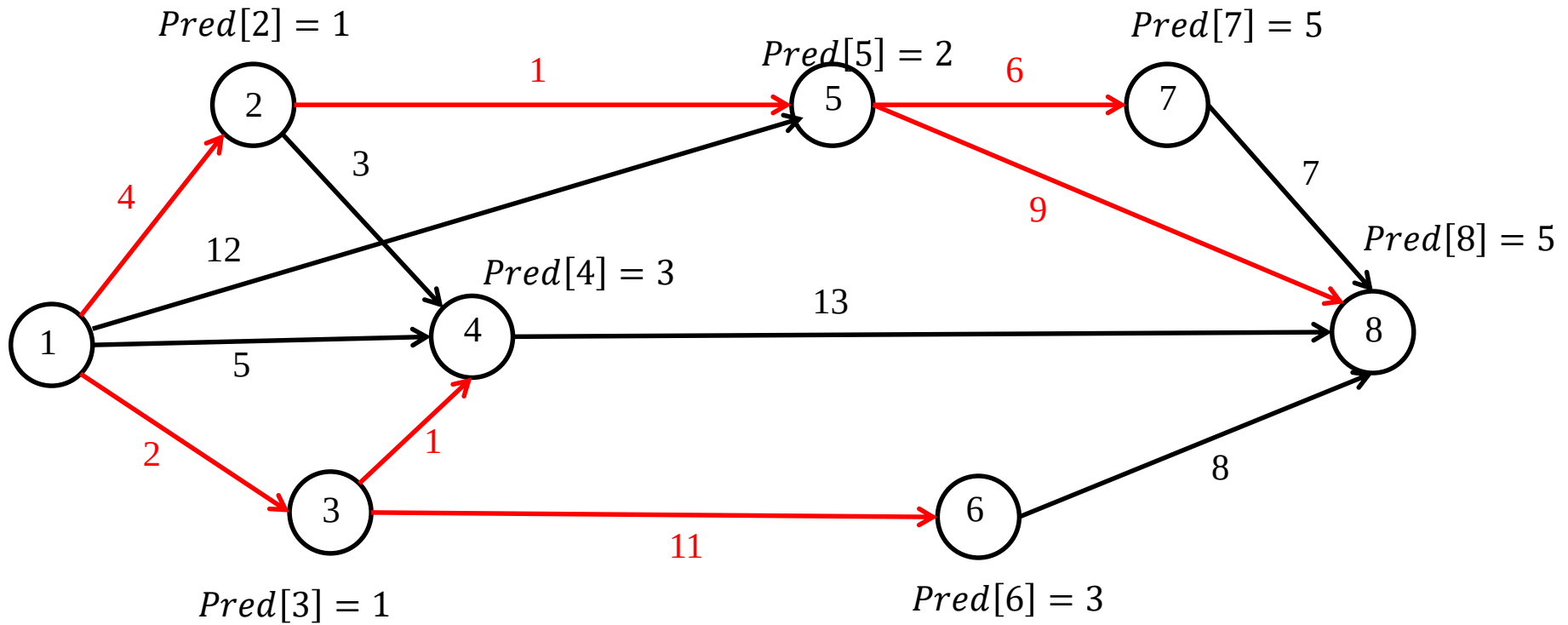
- il cammino di costo minimo dal nodo 1 al nodo 8 vale $L[8] = 14$
- la sequenza di nodi che lo compongono si ottiene ripercorrendo a ritroso l'elenco dei predecessori di ogni nodo a partire dal nodo 8 sino al nodo 1, ossia leggendo in ordine inverso la lista di nodi

8, $Pred[8] = 5$, $Pred[5] = 2$, $Pred[2] = 1$



Analogamente si ottengono i cammini dal nodo 1 a tutti gli altri nodi

Albero dei cammini di costo minimo



Algoritmo di Floyd-Warshall

L'algoritmo di Floyd-Warshall calcola i cammini di costo minimo

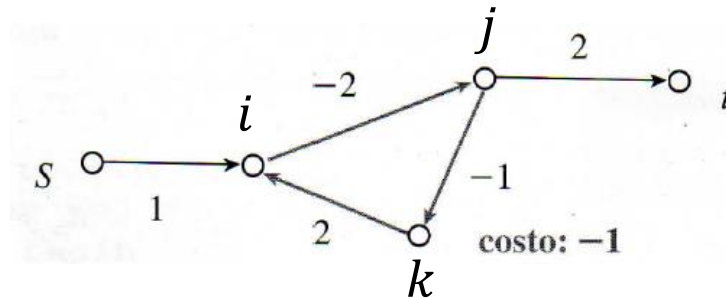
tra ogni coppia di nodi in un grafo generale, ossia anche in grafi contenenti

- archi di costo negativo (caso in cui non è utilizzabile l'algoritmo di Dijkstra)
- cicli (caso in cui non è utilizzabile l'algoritmo con ordinamento topologico dei nodi).

L'algoritmo di Floyd-Warshall ha complessità computazionale maggiore dei precedenti,

verrà utilizzato solo se non sono soddisfatte le ipotesi che rendono utilizzabili i precedenti algoritmi.

Un grafo in cui sono presenti sia **cicli** che **archi di costo negativo** possono contenere **cicli di costo negativo**



Nel caso in figura, il cammino di costo minimo dal nodo s al nodo t è costituito dalla sequenza di nodi

$s, i, j, k, i, j, k, \dots, \dots, \dots, i, j, k, t$

percorrere il ciclo i, j, k infinite volte

Il costo del cammino, variabile obiettivo che deve essere minimizzata, è inferiormente illimitata

$\Rightarrow$ non esiste una soluzione ottima finita

Algoritmo di Floyd-Warshall

Individua l'esistenza di un ciclo di costo negativo e in tal caso
interrompe la procedura di determinazione del cammino di costo minimo.

Input: grafo orientato $G = (N, A)$ e matrice C ($n \times n$) dei costi

All'iterazione k

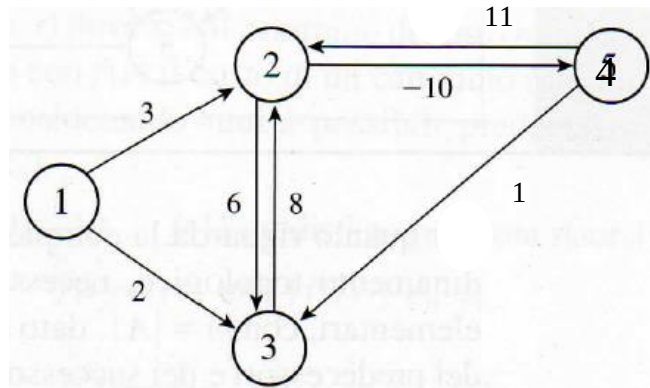
- matrice D , $n \times n$: l'elemento $d_{i,j}$ contiene il costo del cammino più breve finora trovato tra i nodi i e j
- matrice P , $n \times n$: l'elemento $p_{i,j}$ contiene il predecessore del nodo j nel cammino più breve finora trovato tra i nodi i e j

Output:

- elemento $d_{i,j}$: **costo di un cammino minimo** da i a j
- elemento $p_{i,j}$: **predecessore di j nel cammino minimo** da i a j

Esempio

Determinare il cammino di costo minimo dal nodo 1 al nodo 4 nel grafo G in figura.

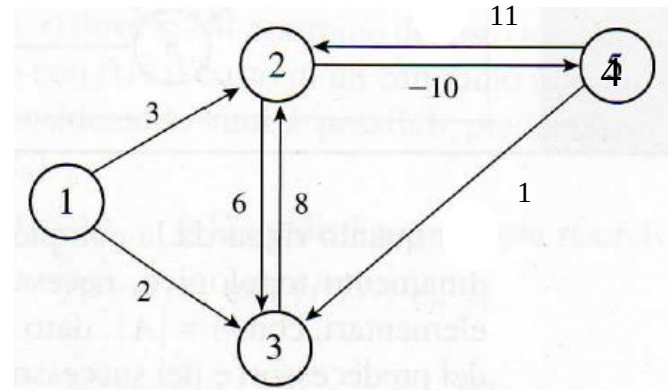


- l'arco (2, 4) ha costo negativo (no Dijkstra)
- è presente il ciclo 2, 4, 3, 2 (no algoritmo con numerazione topologica dei nodi)

⇒ algoritmo di Floyd-Warshall

I costi associati agli archi formano la matrice

$$C = \begin{bmatrix} 0 & 3 & 2 & \infty \\ \infty & 0 & 6 & -10 \\ \infty & 8 & 0 & \infty \\ \infty & 11 & 1 & 0 \end{bmatrix}$$



dove si è posto

- $c_{i,i} = 0$ per ogni $i \in N$
- $c_{i,j} = \infty$ per ogni $(i,j) \notin A$

Inizializzazione

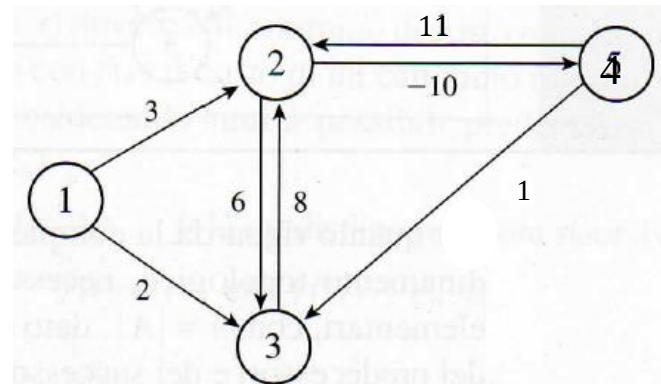
Il cammino più breve tra i nodi i e j è inizialmente posto uguale all'arco che li collega, quindi

- il predecessore del nodo j nel cammino più breve tra i e j è il nodo i : $p_{i,j} = i$

$$P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}$$

- il costo del cammino più breve tra i e j è il costo dell'arco (i,j) : $d_{i,j} = c_{i,j}$

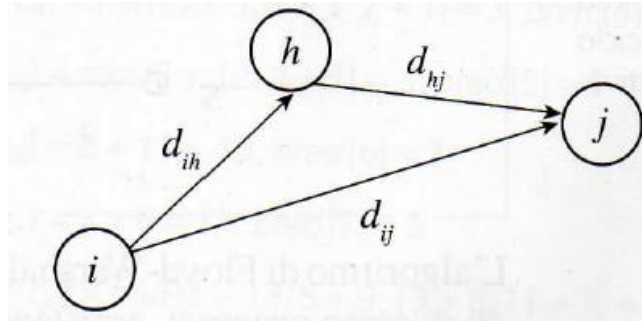
$$D = C = \begin{bmatrix} 0 & 3 & 2 & \infty \\ \infty & 0 & 6 & -10 \\ \infty & 8 & 0 & \infty \\ \infty & 11 & 1 & 0 \end{bmatrix}$$



All'iterazione h viene eseguita la seguente **operazione triangolare** relativa al nodo h :

per ogni coppia di nodi i e j si considera il nuovo cammino ottenuto unendo

- il cammino da i ad h
- il cammino da h a j

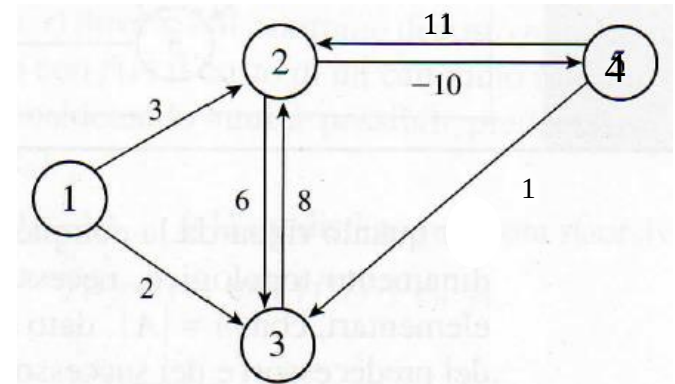


Se il costo del nuovo cammino (passante per h) da i a j è minore del costo del cammino più breve finora trovato, ossia $d_{i,h} + d_{h,j} < d_{i,j}$, si aggiornano sia il predecessore sia il costo di j per identificare il nuovo cammino più breve trovato

- il predecessore di j nel cammino da i viene aggiornato assegnando ad esso il predecessore di j nel cammino da h : $p_{i,j} \leftarrow p_{h,j}$
- il costo del cammino più breve finora trovato da i a j viene sostituito dalla somma dei costi dei cammini più brevi finora trovati da i ad h e da h ad j : $d_{i,h} + d_{h,j} < d_{i,j}$

Iterazione 1: $h = 1$ controllare se $d_{i,h} + d_{h,j} < d_{i,j}$

$d_{1,1} + d_{1,1}$	$d_{1,1} + d_{1,2}$	$d_{1,1} + d_{1,3}$	$d_{1,1} + d_{1,4}$
$d_{2,1} + d_{1,1}$	$d_{2,1} + d_{1,2}$	$d_{2,1} + d_{1,3}$	$d_{2,1} + d_{1,4}$
$d_{3,1} + d_{1,1}$	$d_{3,1} + d_{1,2}$	$d_{3,1} + d_{1,3}$	$d_{3,1} + d_{1,4}$
$d_{4,1} + d_{1,1}$	$d_{4,1} + d_{1,2}$	$d_{4,1} + d_{1,3}$	$d_{4,1} + d_{1,4}$



$0 + 0$	$0 + 3$	$0 + 2$	$0 + \infty$
$\infty + 0$	$\infty + 3$	$\infty + 2$	$\infty + \infty$
$\infty + 0$	$\infty + 3$	$\infty + 2$	$\infty + \infty$
$\infty + 0$	$\infty + 3$	$\infty + 2$	$\infty + \infty$

$$d_{i,h} + d_{h,j} < d_{i,j}$$

?



$D =$

0	3	2	∞
∞	0	6	-10
∞	8	0	∞
∞	11	1	0

Nessuna coppia di nodi (i, j) soddisfa la relazione $d_{i,h} + d_{h,j} < d_{i,j}$

$\Rightarrow$ le matrici D e P sono invariate.

Iterazione 2: $h = 2$

$d_{1,2} + d_{2,1}$	$d_{1,2} + d_{2,2}$	$d_{1,2} + d_{2,3}$	$d_{1,2} + d_{2,4}$
$d_{2,2} + d_{2,1}$	$d_{2,2} + d_{2,2}$	$d_{2,2} + d_{2,3}$	$d_{2,2} + d_{2,4}$
$d_{3,2} + d_{2,1}$	$d_{3,2} + d_{2,2}$	$d_{3,2} + d_{2,3}$	$d_{3,2} + d_{2,4}$
$d_{4,2} + d_{2,1}$	$d_{4,2} + d_{2,2}$	$d_{4,2} + d_{2,3}$	$d_{4,2} + d_{2,4}$

$3 + \infty$	$3 + 0$	$3 + 6$	$3 - 10$
$0 + \infty$	$0 + 0$	$0 + 6$	$0 - 10$
$8 + \infty$	$8 + 0$	$8 + 6$	$8 - 10$
$11 + \infty$	$11 + 0$	$11 + 6$	$11 - 10$

$$d_{i,h} + d_{h,j} < d_{i,j}$$

?



$D =$

0	3	2	∞
∞	0	6	-10
∞	8	0	∞
∞	11	1	0

$D =$

0	3	2	-7
∞	0	6	-10
∞	8	0	-2
∞	11	1	0

$P =$

1	1	1	2
2	2	2	2
3	3	3	2
4	4	4	4

nel cammino minimo da 1 a 4
il predecessore di 4 è 2

nel cammino minimo da 3 a 4
il predecessore di 4 è 2

Iterazione 3: $h = 3$

$d_{1,3} + d_{3,1}$	$d_{1,3} + d_{3,2}$	$d_{1,3} + d_{3,3}$	$d_{1,3} + d_{3,4}$
$d_{2,3} + d_{3,1}$	$d_{2,3} + d_{3,2}$	$d_{2,3} + d_{3,3}$	$d_{2,3} + d_{3,4}$
$d_{3,3} + d_{3,1}$	$d_{3,3} + d_{3,2}$	$d_{3,3} + d_{3,3}$	$d_{3,3} + d_{3,4}$
$d_{4,3} + d_{3,1}$	$d_{4,3} + d_{3,2}$	$d_{4,3} + d_{3,3}$	$d_{4,3} + d_{3,4}$

$2 + \infty$	$2 + 8$	$2 + 0$	$2 - 2$
$6 + \infty$	$6 + 8$	$6 + 0$	$6 - 2$
$0 + \infty$	$0 + 8$	$0 + 0$	$0 - 2$
$1 + \infty$	$1 + 8$	$1 + 0$	$1 - 2$

$$d_{i,h} + d_{h,j} < d_{i,j}$$

?

→

$D =$

0	3	2	-7
∞	0	6	-10
∞	8	0	-2
∞	11	1	0

$D =$

0	3	2	-7
∞	0	6	-10
∞	8	0	-2
∞	9	1	-1

L'algoritmo termina perché l'elemento negativo sulla diagonale di D segnala l'esistenza di un ciclo di costo negativo, da cui segue che il problema è inferiormente illimitato.

Algoritmo Floyd-Warshall (N, A, c)

```
begin
  for  $i = 1$  to  $n$  do
    for  $j = 1$  to  $n$  do
      begin
        porre  $d_{i,j} = c_{i,j}, p_{i,j} = i$  ;
      end
    for  $h = 1$  to  $n$  do \operazione triangolare su  $h$ 
      begin
        for  $i = 1$  to  $n$  do
          for  $j = 1$  to  $n$  do
            if  $d_{i,h} + d_{h,j} < d_{i,j}$  then
              begin
                porre  $d_{i,j} = d_{i,h} + d_{h,j}, p_{i,j} = p_{h,j}$  ;
              end
            for  $i = 1$  to  $n$  do
              if  $d_{i,i} < 0$  then STOP; \ciclo negativo
            end
          end
        end
      end
    end
  end
```

Applicazioni: scheduling delle ispezioni su una linea di produzione

Una linea di produzione è costituita da n celle di lavorazione.

La produzione è attualmente organizzata nel seguente modo:

- nella linea di produzione viene immesso un lotto costituito da B pezzi da lavorare:
i pezzi del lotto sono tutti “senza difetti”, essendo stato effettuato un controllo preventivo nel quale gli eventuali pezzi difettosi sono stati scartati
- in ognuna delle n celle il lotto subisce una lavorazione:
costo di lavorazione nella cella j : p_j per pezzo lavorato
costo della lavorazione del lotto nella cella j : è $B \cdot p_j$,

Al termine della lavorazione n viene effettuata un’ispezione per scartare i pezzi divenuti difettosi nel corso della lavorazione e non consegnarli ai clienti.

Costo totale di produzione di un lotto (lavorazione nelle n celle):

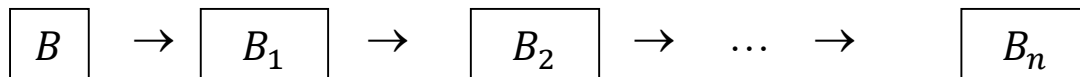
$$B \cdot \sum_{j=1}^n p_j$$

Attualmente la **lavorazione dei pezzi divenuti difettosi continua nelle celle successive** anche se saranno scartati nel corso dell'ispezione al termine della lavorazione n

Un pezzo divenuto difettoso nel corso della lavorazione $j < n$,
viene sottoposto alle lavorazioni da $j + 1$ a n ,
anche se dovrà essere scartato alla fine della lavorazione n .

In alternativa, si possono ispezionare i pezzi lavorati nella cella j , $1 \leq j < n$,
scartando quelli divenuti difettosi e continuando la lavorazione solo sui pezzi non difettosi,
il cui numero è denotato con B_j .

Esempio: con un'ispezione alla fine di ogni lavorazione



si avrebbe $B \geq B_1 \geq B_2 \geq \dots \geq B_n$

Costi delle ispezioni intermedie

Per l'ispezione alla fine della lavorazione j si devono sostenere

- un **costo fisso** (indipendente dal numero degli elementi ispezionati)
- un **costo variabile** (proporzionale al numero degli elementi ispezionati)

In generale, entrambi i costi dell'ispezione alla fine della lavorazione j dipendono da quando è stata effettuata l'ispezione i immediatamente precedente:

$i = 0$: ispezione eseguita al momento della formazione del lotto

$1 \leq i \leq j - 1$: ispezione eseguita al termine della lavorazione i (che precede j)

- $f_{i,j}$, $0 \leq i \leq j - 1$: **costo fisso**
- $h_{i,j}$, $0 \leq i \leq j - 1$: **costo per pezzo ispezionato**

Esempio: le celle di lavorazione sono $n = 4$:

i possibili piani delle ispezioni intermedie sono

1. nessuna ispezione intermedia
2. al termine della lavorazione 1
3. al termine della lavorazione 2
4. al termine della lavorazione 3
5. al termine delle lavorazioni 1 e 2
6. al termine delle lavorazioni 1 e 3
7. al termine delle lavorazioni 2 e 3
8. al termine delle lavorazioni 1, 2 e 3

Numero delle alternative tra cui scegliere: $8 = 2^{n-1}$

$\Rightarrow$ cresce **esponenzialmente** con il numero delle lavorazioni intermedie

Le ispezioni prima dell'inserimento del lotto e alla fine della lavorazione n sono sempre effettuate:

- il costo dell'ispezione prima dell'inserimento del lotto dipende solo da B , quindi, fissato B , è una costante nella funzione dei costi complessivi
- il costo dell'ispezione alla fine della lavorazione n dipende da quando è stata effettuata la precedente ispezione

Problema di segmentazione della sequenza di lavorazioni

Ispezioni intervallate da quattro lavorazioni

0-4

Ispezioni intervallate da tre lavorazioni

0-3

1-4

Ispezioni intervallate da due lavorazioni

0-2

1-3

2-4

Ispezioni intervallate da una lavorazione

0-1

1-2

2-3

3-4

Per ogni possibile piano di ispezione possiamo esprimere il costo totale di produzione+ispezione come somma dei costi relativi ad ogni segmento della lavorazione:

3 termini: costi fissi di ispezione + costi variabili di ispezione + costi di produzione

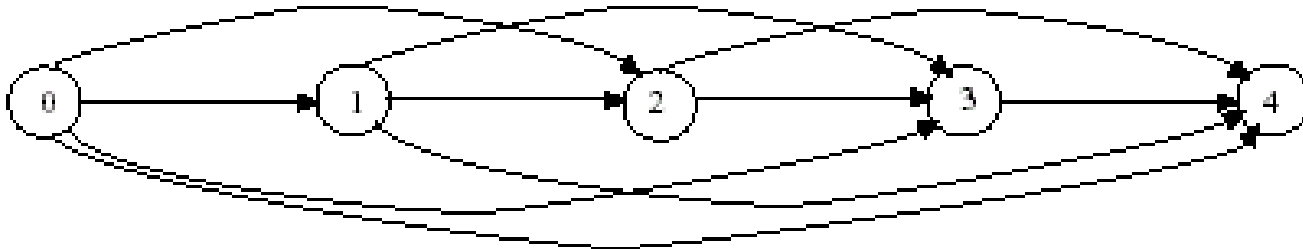
		1	2	3	4
1	0 4				$f_{0,4} + B(h_{0,4} + p_1 + p_2 + p_3 + p_4)$
2	0 1 4	$f_{0,1} + B(h_{0,1} + p_1)$			$f_{1,4} + B_1(h_{1,4} + p_2 + p_3 + p_4)$
3	0 2 4		$f_{0,2} + B(h_{0,2} + p_1 + p_2)$		$f_{2,4} + B_2(h_{2,4} + p_3 + p_4)$
4	0 3 4			$f_{0,3} + B(h_{0,3} + p_1 + p_2 + p_3)$	$f_{3,4} + B_3(h_{3,4} + p_4)$
5	0 1 2 4	$f_{0,1} + B(h_{0,1} + p_1)$	$f_{1,2} + B_1(h_{1,2} + p_2)$		$f_{2,4} + B_2(h_{2,4} + p_3 + p_4)$
6	0 1 3 4	$f_{0,1} + B(h_{0,1} + p_1)$		$f_{1,3} + B_1(h_{1,3} + p_2 + p_3)$	$f_{3,4} + B_3(h_{3,4} + p_4)$
7	0 2 3 4		$f_{0,2} + B(h_{0,2} + p_1 + p_2)$	$f_{2,3} + B_2(h_{2,3} + p_3)$	$f_{3,4} + B_3(h_{3,4} + p_4)$
8	0 1 2 3 4	$f_{0,1} + B(h_{0,1} + p_1)$	$f_{1,2} + B_1(h_{1,2} + p_2)$	$f_{2,3} + B_2(h_{2,3} + p_3)$	$f_{3,4} + B_3(h_{3,4} + p_4)$

n° piani di ispezione: $2^{n-1} = 8$ [esponenziale in n]

$$c_{i,j} = f_{i,j} + B_i \left(h_{i,j} + \sum_{k=i+1}^j p_k \right)$$

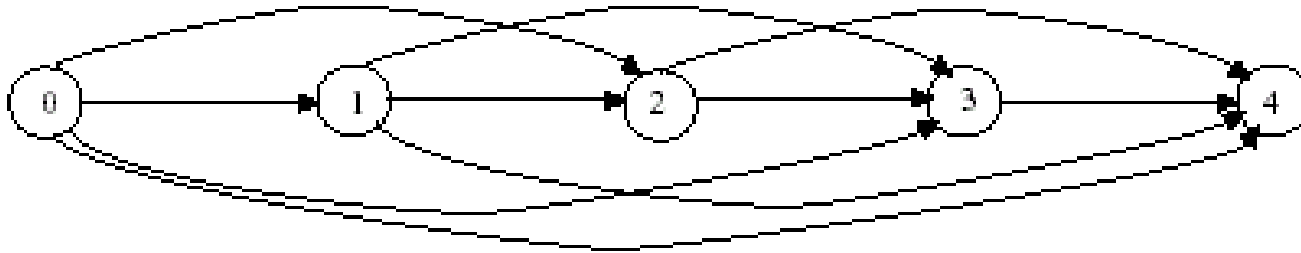
Valutare l'opportunità di effettuare ispezioni intermedie in aggiunta a quella finale, evitando di sottoporre a lavorazione i pezzi divenuti difettosi, al fine di minimizzare i costi totali di produzione e ispezione

- quante ispezioni effettuare?
- alla fine di quali lavorazioni?



Definiamo una variabile binaria per ogni arco:

$$x_{0,1}, x_{0,2}, x_{0,3}, x_{0,4}, x_{1,2}, x_{1,3}, x_{1,4}, x_{2,3}, x_{2,4}, x_{3,4} \in \{0, 1\}$$



Vincoli:	
$x_{0,1} + x_{0,2} + x_{0,3} + x_{0,4} = 1$	si determina la lavorazione al termine della quale effettuare la prima ispezione
$x_{0,1} = x_{1,2} + x_{1,3} + x_{1,4}$	se si effettua un'ispezione al termine della lavorazione 1 , si determina la lavorazione al termine della quale effettuare la successiva ispezione
$x_{0,2} + x_{1,2} = x_{2,3} + x_{2,4}$	se si effettua un'ispezione al termine della lavorazione 2 , si determina la lavorazione al termine della quale effettuare la successiva ispezione
$x_{0,3} + x_{1,3} + x_{2,3} = x_{3,4}$	se si effettua un'ispezione al termine della lavorazione 3 , si determina la lavorazione al termine della quale effettuare la successiva ispezione
$x_{0,4} + x_{1,4} + x_{2,4} + x_{3,4} = 1$	si deve effettuare l'ispezione al termine della lavorazione 4

Il problema del piano di ispezione che minimizza i costi di produzione e ispezione può essere formulato come il seguente modello di programmazione lineare a variabili binarie,

$$\begin{aligned}
 \min \quad & \sum_{i=0}^3 \sum_{j=i+1}^4 c_{i,j} x_{i,j} \\
 \text{s.a} \quad & x_{0,1} + x_{0,2} + x_{0,3} + x_{0,4} = 1 \\
 & x_{0,1} = x_{1,2} + x_{1,3} + x_{1,4} \\
 & x_{0,2} + x_{1,2} = x_{2,3} + x_{2,4} \\
 & x_{0,3} + x_{1,3} + x_{2,3} = x_{3,4} \\
 & x_{0,4} + x_{1,4} + x_{2,4} + x_{3,4} = 1 \\
 & x_{i,j} \in \{0, 1\}
 \end{aligned}$$

dove

$$c_{i,j} = f_{i,j} + B_i \left(h_{i,j} + \sum_{k=i+1}^j p_k \right)$$

Poiché la matrice dei coefficienti dei vincoli funzionali è **totalmente unimodulare** (ossia i determinanti di tutte le sottomatrici della matrice dei coefficienti del sistema dei vincoli), il modello si può esprimere in termini di variabili reali nonnegative

$$x_{0,1}, x_{0,2}, x_{0,3}, x_{0,4}, x_{1,2}, x_{1,3}, x_{1,4}, x_{2,3}, x_{2,4}, x_{3,4} \geq 0$$

- n° variabili decisionali: $n(n+1)/2 = 10$ $[O(n^2/2)]$
- n° vincoli funzionali: $n+1 = 5$ $[O(n)]$
- n° piani di ispezione: $2^{n-1} = 8$ $[esponenziale \text{ in } n]$

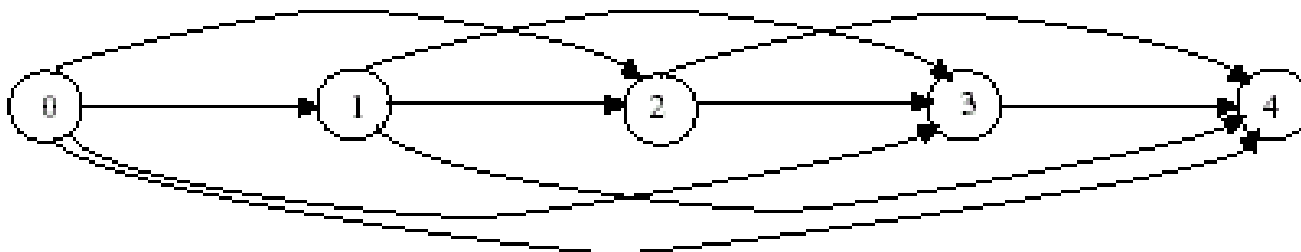
Il problema di determinare il piano di ispezioni ottimo (= quando effettuare le ispezioni)
che minimizza il costo totale di produzione e ispezione
è equivalente al problema di **determinare un cammino di costo minimo**
dal nodo 0 al nodo n sul grafo $G = (N, A)$ in cui

$$N = \{ 0, 1, \dots, n \}$$

$$A = \{ (i, j) : 0 \leq i \leq n-1, i+1 \leq j \leq n \}$$

L'arco (i, j) , con costo $c_{i,j}$, rappresenta il segmento produttivo $(i+1)-j$ che risulta definito se l'ispezione in j è immediatamente consecutiva all'ispezione in i .

Grafo associato al problema delle ispezioni nel caso di $n = 4$:



Corrispondenza biunivoca tra gli elementi degli insiemi

cammini orientati dal nodo 0 al nodo n

piani di ispezione

Costo del cammino orientato (somma dei costi degli archi del cammino)

coincide con il costo globale del corrispondente piano di ispezione

Costo dell'arco (i, j) :

$$c_{i,j} = f_{i,j} + B_i \left(h_{i,j} + \sum_{k=i+1}^j p_k \right)$$

Stima dei valori di B_i , $1 \leq i \leq n$:

- si raccolgono osservazioni per un certo periodo, in base alle quali si stima la probabilità α_i che un pezzo, non difettoso all'inizio della lavorazione i , divenga difettoso nel corso della lavorazione i
- note le probabilità α_i e il numero B di pezzi inizialmente presenti nel lotto, il numero atteso di pezzi non difettosi alla fine della lavorazione i è

$$B_i = B \prod_{k=1}^i (1 - \alpha_k)$$